claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_fc91ae6e-b62\agent-a9a838516d11fe8a2.jsonl`
- SHA-256: `9bf597dbf6084885826e6c6395ffdfa9172aa4a55497d89e7e5e8e9747d4e040`
- Source modified: `2026-06-14T06:52:32+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_fc91ae6e-b62`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-14T06:50:14.732Z)

In the repo (cwd), extract the SYSTEM + METHOD + the measurement-honesty framework. Read docs/CODE_MAP.md, docs/HOW_RALLY_DETECTION_WORKS.md, docs/MULTI_SIGNAL_FUSION_PLAN.md, docs/EXPERIMENT_HARNESS.md, and docs/ANALYZERS_AND_RECONCILERS.md (esp. the §13 literature-review / novelty framing). Return: the 2-layer mental model (shuttle-trajectory substrate -> candidate windows -> learned scorer); how signals are confidence-weighted FEATURE COLUMNS a learned scorer weights (NO special-casing, default-OFF cues, promote-only-on-lift); the measurement-honesty course-corrections C1-C4 (leave-one-video-out / LOVO, bootstrap 95% CI, paired sign test, calibration, F1@k/segment-count-ratio); and the HONEST related-work framing (skeleton = Temporal Action Localization/Segmentation + late/score-level fusion = standard prior art -> "cite, don't claim"; only the report-first reconciler-as-linter rated a novel-combination).

2. assistant (2026-06-14T06:50:26.712Z)

I'll read the five docs to extract the system, method, and measurement-honesty framework. Let me start by pulling the repo and reading all five files in parallel.

3. user (2026-06-14T06:50:29.590Z)

Already up to date.

4. assistant (2026-06-14T06:50:33.635Z)

Let me read all five docs in parallel.

5. user (2026-06-14T06:50:34.535Z)

```
# How rally detection works (the 2-layer mental model)

A short conceptual map of how we go from raw video to rally `[start, end]` segments ?
and, crucially, **which layer our annotation can improve.**

```
  video ??? frames ??? [DETECTION] ??? shuttle (x,y,visible) per frame
                            ?   ?
                       (WASB CNN,   per-frame trajectory
                        frozen weights)   ?
                                                 ?
                                          velocity signal (how fast the shuttle moves)
                                                 ?
                                          [SEGMENTATION] ??? the 3 knobs we tune
                                                 ?
                                                 ?
```
```

```
                                             rally windows  [start, end] ??? IoU vs golden labels
```

Two layers ? keep them separate in your head

1. DETECTION ? "where is the shuttle, this frame?"
WASB is a CNN that, for each frame, outputs a heatmap ? the shuttle's `(x, y)` + a visibility/confidence. Its weights were learned (by gradient descent, on shuttle-coordinate labels) and we use them **frozen**. We chose a *shuttle* detector on purpose: "shuttle in flight" is the signal for "a rally is happening," and it's **camera-agnostic** ? unlike player-motion, which fooled the old heuristic on broadcast cameras that pan with the players. (Your videos are static tripod, so player-motion would also work ? but shuttle-motion is the robust general bet.)

2. SEGMENTATION ? "when is a rally?"
From the per-frame `(x, y)` we compute the shuttle's **velocity** (displacement between frames). Then three knobs turn that into rally intervals:
- **`velocity_thresh`** ? a frame is "active" only if the shuttle is moving faster than this (in flight, not lying on the floor / being picked up).
- **`merge_gap`** ? active frames within this many seconds get joined into one rally (bridges brief gaps: a shuttle momentarily occluded, or the beat between two hits).
- **`min_window_duration`** ? throw away windows shorter than this (blips / false flickers).

Output: a list of `[start, end]` windows = the rallies.

Where the annotation fits (the key point)
Our Roora/golden labels are **rally `[start, end]` times** ? they say *when* rallies are, not *where the shuttle is*. So they can't teach WASB's detector (that needs `(x,y)` labels). They **can** grade and tune the **segmentation layer**: we score predicted windows against the golden rallies with **temporal IoU** (how much they overlap), and pick the 3 knobs that maximise it ? **calibrate on some rallies, measure on held-out ones** so the number is honest.

Where sparks could land (open questions ? not commitments)
Things worth a shower-thought; any of these could beat 3 hand-tuned thresholds:
- **Richer features than raw velocity:** acceleration, trajectory *continuity* (a real rally is a smooth arc that survives occlusion), shuttle *height/arc*, or **direction reversals** (each reversal ? a shot ? ties to `shots_count`!).
- **A tiny learned segmenter** instead of 3 thresholds: feed the trajectory features to a small model trained on the golden rally labels (still the segmentation layer, just smarter).
- **Hysteresis** (two thresholds: enter-rally high, stay-in-rally low) ? robust to mid-rally dips.
- **Rhythm/periodicity:** rallies have a hit cadence; dead time doesn't.
- **Fusion:** combine shuttle-motion (WASB) with player-motion for static cams. ? designed in MULTI_SIGNAL_FUSION_PLAN.md (one rally-fusion layer; the person cue is ADR-007's parked #2 and `rally_gate.py`'s named-but-unbuilt "Gate B").
- **Per-video calibration as a *feature*:** "watch a few labeled rallies ? auto-tune to this

court/lighting ? segment the rest sharply" ? exactly your earlier idea, at the right layer.

The whole point of per-video calibration: a handful of *your* labeled rallies nudges these knobs
to *your* court/camera, and we report the IoU lift on the rallies it never saw.

6. user (2026-06-14T06:50:34.867Z)

Multi-signal rally fusion ? add a human (person) cue alongside the shuttle

Status: **P1 DONE** (2026-06-13) ? the person cue is extracted to
`backend/pipeline/detectors/person_detector.py` (`PersonDetector`), consumed by `yolo_hybrid` with no
behavior change. **P2?P4 remain owner-gated.** Phases below are sequenced but each is a separate PR.
Owner decisions captured in [§9](#9-open-decisions-for-the-owner).

> **Companion docs:** the 2-layer mental model is HOW_RALLY_DETECTION_WORKS.md;
> the modular-fusion architecture this builds on is **ADR-007** (archives/decisions/DECISIONS.md);
> the permissive-dependency rule is **ADR-002 / ADR-005**; the audio cue (the #1 next cue, already
> scaffolded) is archives/research/AUDIO_RALLY_DETECTION_PLAN.md.
> The vendor labels that gate the learned path come from GOLDEN_SET_VENDORING.md
> and the labeling spec in ROORA_LABELING_GUIDELINES.md.
> **How each cue here is added/tested without production regression** ? A/B, shadow eval, and the
> promote-only-on-lift discipline ? is EXPERIMENT_HARNESS.md (every P1?P4 cue
> below lands flag-gated, default OFF, promoted only on measured LOVO lift).

---

1. Context ? why this, why now

Rally detection today runs **one cue at a time**. Each segmenter independently turns a single signal
into rally windows:

Segmenter	Signal	File
`wasb_hybrid` / `tracknet_hybrid`	shuttle velocity (frozen WASB/TrackNet trajectory)	`backend/pipeline/detectors/tracknet_runner.py`
`yolo_hybrid` (the current `default_segmenter`)	person velocity (torchvision + ByteTrack)	`backend/pipeline/segmenters/yolo_hybrid.py`
`motion`	raw pixel motion	`backend/pipeline/segmenters/motion.py`
`gemini`	a cloud VLM watching the whole clip	`backend/providers/gemini.py`

There is no fusion. The owner's directive: make the detector **use the shuttle *and* the humans
together** ? and be ready to fold in more signals (audio, later pose) ? delivered
MIT / Apache-2.0 / BSD-clean so we can ramp commercially without a licensing rethink.

This is **not greenfield** ? it operationalizes work the repo already blessed:

- **`HOW_RALLY_DETECTION_WORKS.md` open question:** *"Fusion: combine shuttle-motion (WASB) with
  player-motion for static cams."*
- **ADR-007** already designed a **modular "rally layer"** that fuses cues *outside* the

frozen WASB model, with **audio as the #1 next cue and person/pose parked as #2**. This plan picks up the parked person cue and generalizes the fusion seam so future cues are "register a producer," not "re-touch the segmenter."
- **`backend/pipeline/detectors/rally_gate.py` already exists** ? the net-crossing gate, explicitly labelled in its own docstring as *"Gate A in the over-fire fix spectrum (**B = player-stance state machine, future**)."* The person cue in this plan **is that named-but-unbuilt Gate B.**

The owner's "held-shuttle" false positive ? what's actually going on

The owner observed a "rally" that was just a player **holding / flicking the shuttle in hand** between points. This is the *exact* failure `rally_gate.py` was written to kill:

> *"WASB over-detects: when a player flicks/tosses the shuttle back to the opponent for service after
> losing a point, that single trajectory looks like a rally. The discriminator is structural: a real
> rally has the shuttle crossing the net MULTIPLE times; a single service feed crosses ~once."*

The structural fix already exists and is validated offline ? but it is NOT wired into the production runtime path. `rally_gate.apply_gate(..., min_crossings=2)` is called only from the **eval drivers** (`distill_local.py`, `eval_partial_labels.py`, `export_wasb_segments.py`, `calibrate_local.py`); there is **no `min_crossings` knob in `IndexingConfig`** and the live segmenters don't apply it. So the single cheapest, highest-confidence win in this whole plan is **P2's first step: fold the existing Gate A into production.** Everything else (the person cue / Gate B, the learned fusion) is robustness on top of that.

Why humans help ? and the honest risk

The beachhead videos are **static tripod** (Bangalore academies, PMF_MARKET_RESEARCH.md), so player presence/motion is a usable cue. ADR-001's warning was specifically about **broadcast pan** cameras, where player-motion fooled the old heuristic ? *that failure mode does not apply to a fixed court cam.* But we keep **the shuttle as the camera-invariant primary** so the general bet stays robust on other camera types. Humans then add:

- **Precision** ? reject shuttle-on-floor / warm-up knock-ups / held-shuttle feeds when the court isn't occupied by ?2 players in a play posture.
- **Recall** (via the learned classifier) ? features that help bridge shuttle occlusion/blur gaps.

---

2. Core architecture ? one rally-fusion layer, cues are swappable producers

Formalize ADR-007's diagram. Every cue is a **black box that emits a time-aligned signal** (per-frame activity and/or per-window features); **fusion happens in *our* layer, never inside a frozen model.**

```
frames ?? WASB (frozen, MIT)      ?? shuttle trajectory ???
frames ?? person detector (ours)  ?? tracks + boxes ???????
audio  ?? hit detector (ours)     ?? hit events ????????????? RALLY LAYER (ours) ?? rallies
[frames ?? pose/stance (ours; parked #3) ?? stance ????????   feature + decision fusion
```

- **Shuttle = primary, camera-invariant.** Windows still **seed** from `TrackNetRunner.trajectory_to_action_windows()` (`tracknet_runner.py:234`). Other cues *enrich and adjudicate* shuttle-seeded windows; they do **not** replace the shuttle seed. Person runs only over **shuttle-seeded candidate windows + padding**, not the whole video ? which bounds the added cost.
- **A small `RallyCue` contract.** Each producer yields `(frame_idx ? features)` + per-window stats. Adding a cue = register a producer; the existing `SegmenterRegistry` (`segmenters/base.py:35`) is the template for the pattern. Keep the single-cue segmenters registered as fallbacks.

---

3. The two fusion modes (both ? per the owner decision)

3.1 Decision-level gate (safe baseline, ships first, no labels needed)

```
rally_active = shuttle_motion ? (?2 persons in court zone ? recent person track)
```

The `? recent-track` clause tolerates 1?2 frame person-detector misses so the AND-gate can't cause a recall cliff. This composes with the **existing net-crossing Gate A**:

```
keep window  ?  net_crossings ? 2            (Gate A ? EXISTS in rally_gate.py, eval-only today)
              ?  (?2 in-court players ? recent track)   (Gate B ? the person cue, NEW)
```

Gate A kills the service-feed / held-shuttle case structurally; Gate B adds court-occupancy evidence for the cases a single trajectory can still fool (e.g. a knock-up that does cross the net). No training data required ? this is the label-free fusion that ships before any golden labels exist.

3.2 Feature-level (primary, the learned path)

The cues emit a **small set of aggregated, scale/translation-invariant per-window features ? NOT raw coordinates.** Raw pixel coordinates would memorize *this* court/camera and fail to generalize off a still-small labeled set; counts / ratios / zone-membership / two-sidedness transfer.

What already feeds the classifier (`distill_local.py:40`, `FEATURE_NAMES`): `duration, peak_velocity, mean_velocity, active_ratio, net_crossings` ? note **`net_crossings` is already a feature** (computed via `rally_gate.gate_windows`).

Person features to add (~4, deliberately few):

| Feature | Meaning | Rally signal |

|---|---|---|
`players_in_court`	median count of persistent in-court tracks	? 2 (singles) / 4 (doubles); 0?1 = dead time
`two_sided_motion`	are moving players on **both** net-halves?	rally = two-sided; one person feeding = one-sided
`mean_player_motion`	aggregate normalized player velocity	rally = sustained; resting/standing = low
`in_court_ratio`	fraction of frames with ?2 in-court players	track stability vs flicker
`shuttle_player_colocation`	fraction of frames the shuttle sits inside/adjacent to a person box (hand region)	**held = high; in-flight = mostly free space** ? the direct held-shuttle discriminator

These join the shuttle features in the candidate `stats` JSON (`database.py::add_candidate_segment`) and the feature vector of the **existing distilled classifier ? the tiny numpy-only L2 logistic regression in `backend/eval/classifier.py` (ADR-004), NOT a new net.** It scores each candidate window `P(real rally)` and replaces the Gemini call at runtime.

What the model learns: the **weights** on those few features ? e.g. down-weight a shuttle-motion window with 0?1 in-court players / one-sided motion / high shuttle-colocation (warm-up, floor shuttle, held-shuttle feed); up-weight ?2 players, two-sided, sustained motion, shuttle in free flight.

Small-N discipline (non-negotiable):
- Keep to **~4 person features** ? more features on a still-small labeled set overfit; that's exactly why the model stays a logistic regression, not a net.
- Use the **leave-one-video-out (LOVO)** protocol that `distill_local.py` **already implements**: train on the other videos' candidates, predict the held-out *video*, score vs its labels ? never a held-out *window* from the same video (windows in one video are correlated; that flatters the score).
- Training labels today are **Gemini silver labels** (distilled offline, ADR-004/007 ? Gemini is a teacher, never a runtime dependency, never a training-data *source* for owned weights); **golden labels are the honest measuring stick**, and as the Roora golden corpus grows they can also become training labels. Until the labeled set is larger, treat the learned weights as **calibration / direction-check, not "a fully general model"** ? the decision-level gate (§3.1) is the shippable, label-free fusion in the meantime.

> *(Gated on golden labels existing ? the same blocker ADR-007 flags for audio. See
> GOLDEN_SET_VENDORING.md.)*

---

4. Shuttle-state: in-flight vs in-hand (the misclassification, mapped to real code)

The "shuttle moved" seed conflates a shuttle **in flight** with a shuttle **carried in hand**. The discriminators, and where each lives:

Signal	In-flight	Held / static	Status
`net_crossings`	? 1 (usually several)	~0 (held) / ~1 (single feed)	**EXISTS** (`rally_gate.py`); eval-only ? wire to prod
`shuttle_player_colocation`	low (free space)	high (near a hand)	**NEW** (needs the person cue)

`direction_reversals`	several (racket contacts)	~0	derivable from trajectory; partial overlap with crossings
`peak shuttle speed`	high	walking speed	derivable from existing velocity

So the in-flight-vs-held fix is **mostly Gate A (already built) + the new colocation feature from the person cue**. **No new label column is needed** ? the golden labels already start a rally at the *serve-contact frame* and treat all pre-serve / held-shuttle time as dead time (see ROORA_LABELING_GUIDELINES.md), so held-shuttle moments are negatives by construction and are the ground truth that proves the detector wrong.

---

5. Adversarial mitigations (integrating safely)

- **Court-region mask** ? polygon around the court; drop persons outside it (spectators, coaches, adjacent courts). Config-supplied per camera; ~1 ms. Source: a manual per-camera polygon first; the **4 court corners + net-line** that Roora labels once per video (see the labeling doc) build this mask **and** the net split that `two_sided_motion` / `net_crossings` need.
- **Player-count constraint** ? expect **2 (singles) / 4 (doubles)** persistent in-court tracks; suppress transient/extra detections. The `singles_or_doubles` manifest flag sets the expectation.
- **Temporal association** ? reuse `supervision` **ByteTrack** (already in `yolo_hybrid`); reject singleton detections; smooth across frames.
- **Multi-court rejection** ? proximity to the primary court region (homography optional, later).
- **Domain shift (COCO frontal ? side/overhead court)** ? start **box-only** (robust); keep the detector swappable. Fine-tuning a person detector is **parked** ? we have no in-house person labels, and the **paid-LLM/training guardrail + minors-exclusion** apply to any future person-training data.
- **Fusion-degradation guard** ? shuttle stays primary; person is gate + feature, never the sole trigger; the single-cue segmenters remain registered fallbacks.

---

6. Box vs pose

Box-first. Occupancy + count + two-sided motion + shuttle-colocation is enough for both the gate and the features, and box detection is the cheap, robust win. **2D pose is parked behind the same `RallyCue` interface** ? pose (MediaPipe Pose / RTMPose, both Apache-2.0) buys serve-ready stance gating and shot cues later, at the cost of a second model. This matches ADR-007 parking pose as the #2/#3 cue.

---

7. Performance

The person detector is the **existing in-process torchvision model** run at **frame-skip 3?5** (`yolo_frame_skip` default 3) with track interpolation between sampled frames. The shuttle keeps

running in its separate WSL process ? the two are **separate processes**, so a "shared backbone" isn't
free; budget realistically as the existing `yolo_hybrid` cost minus skip savings. Person runs **only
over shuttle-seeded windows + `ai_handoff_padding`**, not the whole video, which bounds the cost.

---

8. Licensing (the guardrail table ? all green)

Every dependency stays **MIT / Apache-2.0 / BSD** (ADR-002 / ADR-005; `ultralytics` was dropped for
AGPL-3.0).

Cue / model	Code	Weights / data	Verdict
Shuttle: WASB-SBDT, TrackNetV3	MIT	frozen badminton weights	? (ADR-001/002)
Person (default): torchvision Faster R-CNN / RetinaNet / FCOS	BSD/MIT	COCO-pretrained	? already in repo (`_DETECTOR_FACTORIES`)
Person (escalation): **YOLOX**, **RT-DETR / RT-DETRv2**	Apache-2.0	COCO / Objects365	?
Pose (parked): **MediaPipe Pose**, **RTMPose / MMPose**	Apache-2.0	COCO-structure	?
Tracking: `supervision` ByteTrack	MIT	n/a	? already in repo
Banned	**Ultralytics YOLOv5/8/11 (AGPL-3.0)** · **MonoTrack (Adobe, non-comm)** · **YOLO-NAS (Deci, non-comm)**	?	?

COCO caveat to record: COCO *annotations* are **CC BY 4.0** (commercial OK *with attribution*);
COCO *images* are Flickr-ToS (per-image). Pretrained-**weight** use is clean; record the attribution.

---

9. Open decisions (for the owner)

1. **Court-region mask source** ? manual per-camera polygon (simple, ship first) vs auto court
   homography/line detection (later). **Recommend manual first**, fed by Roora's court-corner labels.
2. **Person detector default** ? keep torchvision (zero new deps, BSD/MIT) vs adopt YOLOX/RT-DETR
   (Apache, faster) now. **Recommend torchvision first**, escalate only if eval shows a speed/accuracy gap.
3. **Pose now or parked** ? **recommend parked** behind the interface (box-first).
4. **Golden labels** ? feature-level fusion (P3) is **blocked on golden rally labels**; the source is
   the VLC `rally-annotator` + the Roora vendor pilot (GOLDEN_SET_VENDORING.md).

---

10. Phased execution (owner-gated; ONE PR per slice, off `master`)

- **P0 ? this design doc** + ROORA_LABELING_GUIDELINES.md, indexed in
  `docs/README.md`, cross-linked from `HOW_RALLY_DETECTION_WORKS.md` and ADR-007. *(this deliverable;
  docs-only, no code.)*
- **P1 ? extract the shared person cue:** ? **DONE (2026-06-13).** Lifted the torchvision detector +
  ByteTrack + velocity out of `yolo_hybrid` into `backend/pipeline/detectors/person_detector.py`
  (`PersonDetector`); `yolo_hybrid` now orchestrates only and delegates to `self.person`

(no behavior change ? regression-safe, proven by the unchanged process_video output tests). New `tests/test_person_detector.py`; suite +1 green; person_detector is type-clean (left mypy quarantine for the moved code). The fusion segmenter (P2) reuses this same producer over shuttle-seeded windows.
- **P2 ? the FusionSegmenter (Gate A + Gate B + person-feature persistence):** ? **DONE (2026-06-14).** Shipped `backend/pipeline/segmenters/fusion_hybrid.py::FusionSegmenter` (registry `fusion_hybrid`, **default-OFF** ? `default_segmenter` unchanged), subclassing the trajectory engine via two identity hooks (`_enrich_candidate_stats`, `_select_for_handoff` ? wasb/tracknet stay byte-identical, adversarially verified). It seeds windows from the shuttle substrate, **always persists `net_crossings`** (Gate A signal), and ? when `indexing.fusion.cue_flags.person` is on ? runs the person cue over each window's frames (`PersonDetector.detections_per_frame`) and persists the ~5 person features (`fusion_features.py`, pure + unit-tested). Served gating is optional + default-OFF: `fusion.min_crossings` (Gate A) / `fusion.min_players` (Gate B) skip sub-threshold windows from the handoff while **persisting the full superset** (offline substrate intact). Court mask = optional `fusion.court_polygon` (None ? count-only). Lift to be measured via the harness; held-shuttle-drop + person-feature values are GPU/owner-side.
- **P3 ? feature-level (learned) fusion:** add the person features (incl. `shuttle_player_colocation`) to `distill_local` / `classifier`; recalibrate IoU on the golden set; report **LOVO lift on held-out videos**. *(Gated on golden labels.)*
- **P4 ? additional cues:** wire **audio** (ADR-007's #1, already scaffolded in `backend/pipeline/audio/hit_detector.py`) and later **pose**, through the same `RallyCue` interface. Each is add-a-producer, not a refactor.

---

11. Verification

- **Offline, runnable in this dev container (no GPU/torch/cv2):** unit tests for the fusion gate logic and `RallyCue` aggregation (court-mask filter, player-count constraint, `? recent-track` tolerance) and the shuttle-state signals (net-crossing count ? extends the existing `rally_gate` tests ? and person-colocation) with **synthetic trajectories + person boxes**. A key-license assertion test (no `ultralytics` import; detector factories stay torchvision/Apache).
- **Machine-side (owner, GPU/WSL ? the full pipeline can't run in this container):** run the `FusionSegmenter` on a real static-tripod clip; confirm **held-shuttle / warm-up / spectator false positives drop** vs shuttle-only (this should reproduce the owner's observed bug and show it fixed ? start by confirming Gate-A-in-production alone fixes it); **LOVO IoU lift on the golden set** for the learned path.
- **Docs (P0):** links resolve; both docs indexed in `docs/README.md`; this doc cross-linked from `HOW_RALLY_DETECTION_WORKS.md` (the fusion open-question) and ADR-007.

7. assistant (2026-06-14T06:50:38.510Z)

Now let me read the remaining three docs.

8. user (2026-06-14T06:50:39.484Z)

```
1	# Experiment harness ? A/B + shadow eval for rally-detection quality (no-regression by design)
2
3	**Status:** **P1 IMPLEMENTED** (`backend/eval/experiment.py` + `tests/test_experiment.py`, shipped
4	2026-06-13) ? the `Variant` layer, `compare()` (labeled A/B, LOVO-honest), `compare_unlabeled()`
5	(SxS on NEW unlabeled footage), and the leaderboard. **P2?P4 remain owner-gated.** Phases below are
6	sequenced but each is a separate PR.
7
8	> **Companion docs:** the cues this experiments *on* are in
9	> [MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md); the scoring mechanics are
10	> [EVALUATION.md](EVALUATION.md); the no-regression gate + golden labels + `baseline.json` come from
11	> [GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md); the modular-cue architecture is **ADR-007**
12	> ([archives/decisions/DECISIONS.md](archives/decisions/DECISIONS.md)); the distilled classifier is
13	> **ADR-004**. The 2-layer mental model is [HOW_RALLY_DETECTION_WORKS.md](HOW_RALLY_DETECTION_WORKS.md).
14
15	---
16
17	## 1. Context ? why this, why now
18
19	PR #112 (merged) added the multi-signal fusion design ? shuttle + person + audio cues, all **default
20	OFF**. The owner's directive for *how* we evolve quality from here:
21
22	> *"Evolve the codebase so we can keep finding new ways to improve quality and experiment with them ?
23	> or add them as a signal ? without suffering any regressions if the idea doesn't work out. Rolling
24	> back the code would be too dangerous."*
25
26	So **experimentation and deployment must be decoupled.** The contract:
27
28	1. Merge experimental code freely ? it's **gated OFF**, so it cannot change production behavior.
29	2. Test it **offline against golden labels** (most experiments never touch the served path at all).
30	3. Promote a variant to default **only on measured lift**, through a CI eval gate.
31	4. **"Rollback" = flip a config flag, never `git revert`.** The proven path is a pinned variant that is
32	   always present.
33
34	**The pleasant surprise (Explore audit, 2026-06-13):** ~80% of this harness already exists and is
35	simply not composed *as a system*. The missing piece is a thin **variant/experiment layer + the
36	discipline**, not a platform. §4 is the exact code surface so the next session does not re-discover it.
37
38	---
39
40	## 2. The thesis ? separate expensive DETECTION from cheap DECISION
41
42	The expensive, GPU/WSL-bound, risky part is **detection**: "what signals exist per
```

frame" (shuttle WASB trajectory, person tracks/boxes, audio hits). The cheap, swappable, pure-Python part is the **decision**: "given those signals, where are the rallies" (windowing + gates + the classifier).

> **Run detection once per video, persist all per-cue signals + per-candidate features, then re-score
> any number of variants OFFLINE ? deterministically, with no GPU and zero production risk.**

This is not aspirational ? `distill_local.py` and `calibrate_local.py` already do exactly this on stored WASB trajectories. The candidate-segments table (`stats` JSON) is the persistence substrate. ? Most experiments are pure re-scoring of stored data; they never enter the served pipeline.

---

3. What already exists (the audit ? do NOT rebuild)

Exact public surface, with `file:line`. **This table is the anti-duplicate-work artifact** ? start here.

3.1 Scoring (variant-agnostic ? grades any `List[Interval]`)
`backend/eval/metrics.py`
- `interval_iou(a, b) -> float` (:15) ? temporal IoU of two `(start,end)` intervals.
- `match_intervals(preds, gts, iou_threshold=0.5) -> MatchResult` (:48) ? greedy 1-to-1 IoU matching.
- `score(preds, gts, iou_threshold=0.5, match_result=None) -> Score` (:104) ? P/R/F1, mean IoU, boundary errors; `Score.as_dict()`.
- `pr_curve(preds, gts, thresholds=None) -> List[Score]` (:138).

3.2 A/B + cross-validation primitives (the comparison engine already exists)
`backend/eval/calibration.py`
- `improvement_report(predict_fn, baseline_config, tuned_config, gts, iou_threshold=0.5) -> dict` (:148) ? **before/after metrics + %?. This IS the A/B delta.**
- `leave_one_video_out(videos, configs, metric="f1", iou_threshold=0.5) -> dict` (:113) ? **LOVO: pick on other videos, score on held-out video (honest cross-video).**
- `cross_validate(predict_fn, configs, gts, k=5, metric="recall", iou_threshold=0.5) -> dict` (:72) ? within-video k-fold overfit guard.
- `select_best(predict_fn, configs, gts, metric="f1", iou_threshold=0.5) -> (Config, float)` (:61).
- `evaluate(preds, gts, iou_threshold=0.5) -> Score` (:41); `kfold_indices(n, k)` (:46).
- LOVO video dict shape: `{"name": str, "predict_fn": PredictFn, "gts": [Interval]}`.

3.3 No-regression gate + baselines (the promotion gate already exists)
`backend/eval/regression.py`
- `regress(db, video_id, ground_truth, stage="final", iou_threshold=0.5, detector=None, tolerance=DEFAULT_TOLERANCE, baseline_path=DEFAULT_BASELINE_PATH) -> RegressionResult` (:104) ? score stored preds vs GT, flag if P **or** R drops > tolerance.
- `regress_with_provider(...)` (:160) ? same, fetching GT via the annotation provider.
- `save_baseline(video_id, stage, precision, recall, f1, ..., gt_fingerprint=None)` (:80) · `load_baselines(path) -> dict` (:68).
- `RegressionResult` (:43): `regressed`, `reasons`, `gt_hash`, baseline P/R, `tolerance`, `has_baseline`; `as_dict()`.
- Default baseline file: `eval_baselines/regression_baseline.json` (:33).

`run_regression.py` ? CI-gatable CLI, `main(argv)` (:52). **Exit codes: 0 ok / 1 regression / 2 no-DB / 3 no-baseline.** Flags: `--video-id --tier --annotations --stage{candidates,final} --detector --iou --tolerance --baseline --update-baseline --allow-missing-baseline --json`.

3.4 Pinnable model artifact

`backend/eval/classifier.py` ? `LogisticRegression` (pure numpy, L2): `fit(X,y,feature_names=,class_weight="balanced")` (:40), `predict_proba` (:75), `predict(threshold=0.5)` (:81), `to_dict`/`from_dict` (:86/:97), `save`/`load` (JSON) (:107/:111). **A trained model = one JSON file ? a pinnable, hashable variant artifact.**

3.5 Offline re-scoring substrate (persist once, re-score forever)
`backend/infrastructure/database.py`
- `add_candidate_segment(video_id, detector, start_time, end_time, chunk_index=None, confidence=None, stats=None, detection_params=None) -> Optional[int]` (:240) ? `stats`/`detection_params` stored as JSON; `candidate_segments` table cols incl. `stats`, `detection_params`, `human_label`, `confirmed_segment_id` (schema :97).
- `query_candidate_segments(video_id, detector=None, only_unlabeled=False) -> List[dict]` (:277) ? returns rows with deserialized `stats`.

3.6 Label assignment + feature glue
`backend/eval/training.py`
- `label_candidates(candidate_intervals, gt_intervals, iou_threshold=0.5) -> List[int]` (:50) ? y-labels by IoU match to golden (same matcher as the evaluator).
- `build_training_set(candidate_rows, gt_intervals, iou_threshold=0.5, feature_fn=extract_yolo_features) -> (X, y, intervals)` (:64) ? pluggable `feature_fn`.

3.7 Existing LOVO drivers to mirror (not duplicate)
- `backend/eval/distill_local.py` ? `run(specs, iou=0.5, threshold=0.5, model_out=None)` (:98), CLI `python -m backend.eval.distill_local --manifest videos.json --model-out ...`; `FEATURE_NAMES = [duration, peak_velocity, mean_velocity, active_ratio, net_crossings]` (:40); already does LOVO over Gemini silver labels.
- `backend/eval/calibrate_local.py` ? `run(specs, iou=0.5, metric="f1")` (:77); `LocalConfig = (velocity_thresh, merge_gap, min_window_duration, min_crossings)`; grid + LOVO.
- `backend/eval/calibrate_wasb.py` ? `run(...)`, `load_golden_gts()`; `backend/eval/gt_loader.py::load_gt_intervals(csv_path, *, strict=True)` is THE canonical golden reader.

3.8 Registries + config knobs (variant selection)
- `backend/pipeline/segmenters/base.py` ? `SegmenterRegistry` (:35), singleton `segmenter_registry` (:63); registered: `motion`, `gemini`, `yolo_hybrid`, `tracknet_hybrid`, `wasb_hybrid`.
- `backend/annotations/base.py` ? `AnnotationProviderRegistry`, singleton `annotation_registry`; providers `file`, `auto`.
- `backend/config/models.py::IndexingConfig` ? `default_segmenter` (:65, default `"yolo_hybrid"`), `detector_model` (:72), `detector_score_threshold` (:73), `yolo_velocity_threshold` (:74), `yolo_frame_skip` (:75), `min_segment_duration` (:68), `dead_time_merge_gap` (:69), `motion_threshold` (:67). **No `min_crossings` knob yet** (see fusion P2).

3.9 Confirmed ABSENT (no duplication risk)
Explore found **no** existing experiment / variant / shadow / A-B *machinery* ? only a stray "A/B test" aspiration comment in `backend/pipeline/detectors/base.py` and an "experiment E1" note in `AUDIO_RALLY_DETECTION_PLAN.md`. The `regress()` + baseline path is the canonical comparison mechanism.

---

4. The variant abstraction (the one thin thing to add)

A **`Variant` = a declarative recipe**, not a code branch:

```
Variant = (segmenter_name, IndexingConfig overrides, enabled cues + per-cue params, classifier model path/hash)
```

JSON-serializable; selected via the existing `segmenter_registry` + `IndexingConfig.default_segmenter`.

Control = a *pinned, frozen* variant (recipe + model hash) that is always registered and is the default. Adding a new cue = a new Variant differing by one flag ? the control recipe is untouched.

---

5. Three experiment modes

5.1 Offline A/B (primary, label-driven, zero production risk)
`compare(videos, variant_A, variant_B)` ? a thin driver composing **existing** functions: `query_candidate_segments()` (load persisted features) ? re-score each variant ? `calibration.improvement_report()` + `calibration.leave_one_video_out()` + `metrics.score()`, graded vs golden (`gt_loader.load_gt_intervals`). Reports per-video **and** LOVO-honest aggregate IoU/P/R/F1 deltas. **This is the A/B test, and it needs almost no new scoring code** ? only the glue.

Honest reporting (default ON, additive): `compare(..., honest_stats=True)` also attaches a `"honest"` block ? per-metric bootstrap **CIs**, a paired **?F1 CI + exact sign test** (C1), and **F1@k + segment-count ratio** (C4) ? *alongside* the bare-mean fields (existing output unchanged; `honest_stats=False` restores the legacy shape). A bare LOVO mean at n?6 is not promotable on its own; the lift is real when the ?F1 CI clears 0 **and** the sign test agrees. Wiring of the course corrections in ANALYZERS_AND_RECONCILERS.md §13/C1+C4 (helpers: `backend/eval/eval_stats.py`, `segmentation_metrics.py`); `record_experiment` carries the honest ?F1 into the leaderboard.

5.2 Shadow (for cues that touch detection, e.g. the person detector)
The new cue runs and **persists its signal** into the candidate `stats`/`detection_params`, but the *served* decision stays control. Offline you then ask "what would variant B have scored?" via §5.1 ? B never affects output until it wins. This is how a detection-touching change (not just a re-scoring change) earns its way in without risk.

5.3 Promotion gate (no-regression, CI-enforced)
`regression.regress()` + `baseline.json` + `run_regression.py` exit codes. A variant becomes the new default **only if** it doesn't regress the golden set beyond `tolerance` (exit 1 blocks). Control is pinned ? **rollback = flip `default_segmenter`/variant config, never a code revert.**

---

6. No-regression guarantees (the heart of the owner's ask)

- **New cues/signals default OFF** ? per-cue enable flag in `IndexingConfig`; merged code can't change behavior until explicitly enabled.
- **Control variant pinned** ? frozen recipe + classifier model hash, always registered & default.
- **Promotion only via the eval gate** ? `run_regression.py` in CI (exit 1 blocks); no silent default change.
- **Experiments are records** ? variant-spec hash ? per-video + LOVO scores + label-set hash + timestamp, persisted to a small JSON **experiment leaderboard** (generalizes `baseline.json`). Answers "which

signals ever helped, on which video slices," and makes every result reproducible.
- **Decoupled events** ? "merge an experiment" and "change the default" are separate, independently revertible actions.

---

7. What to build later (gap list ? thin, additive, owner-gated)

1. ~~**`backend/eval/experiment.py`** ? a `Variant` dataclass + `compare(videos, a, b)` composing the §3 functions (no new scoring math).~~ **DONE (2026-06-13).** Shipped `Variant` (+ pinned `CONTROL`), `compare` (labeled A/B with LOVO-honest learned-variant training, mirroring `distill_local`), `compare_unlabeled` (the **SxS-on-new-videos** mode ? agreed / A-only / B-only via `match_intervals`, no GT), `record_experiment` (? `eval_baselines/experiment_leaderboard.json`), and `load_video_candidates` (the `query_candidate_segments` bridge). JSON variant recipes first as recommended; a `VariantRegistry` only if the count grows. 19 tests incl. the no-regression invariant.
2. **Persist per-cue features** into candidate `stats` (person: `players_in_court`, `two_sided_motion`, `in_court_ratio`, `shuttle_player_colocation`; shuttle-state: `net_crossings`, `direction_reversals`; audio: hit times) so any future variant re-scores offline ? ties to `MULTI_SIGNAL_FUSION_PLAN.md` §3.2/§4.
3. **Per-cue enable flags** in `IndexingConfig` (default `False`); `default_segmenter` already exists. Add the `min_crossings` knob (fusion P2) here too.
4. **CI promotion lane** using `run_regression.py` against the golden `baseline.json`; write the experiment-leaderboard JSON.

---

8. Phasing (owner-gated; ONE PR per slice, off `master`)

- **P0 ? this doc**, indexed in `docs/README.md`, cross-linked from `MULTI_SIGNAL_FUSION_PLAN.md` and `GOLDEN_SET_VENDORING.md`. *(this deliverable; docs-only, no code.)*
- **P1 ? `experiment.py` `compare()`** ? **DONE (2026-06-13):** thin wrapper over `calibration` + `metrics` + `query_candidate_segments`; pure-logic, fully tested in this dev container (`compare`, `compare_unlabeled`, `record_experiment`, the no-regression invariant). `compare_unlabeled` adds the SxS read on **new unlabeled footage** the owner asked for (run control + candidate on a fresh video, diff where they agree/diverge ? two reels via the existing `VideoCompiler` is the GPU-side follow-on).
- **P2 ? per-cue enable flags + feature persistence** into candidate `stats` (default OFF) ? couples with fusion P1/P2.
- **P3 ? CI promotion gate**: `run_regression.py` lane on the golden `baseline.json`; experiment-leaderboard JSON.

---

9. Scope guardrail

Single-tenant / local-first (per `PLATFORM_ARCHITECTURE.md`): a JSON experiment registry + the existing harness ? explicitly **NOT** MLflow / W&B / a service. Keep it the size of

```
`baseline.json`.

---

## 10. Verification

- **Offline, runnable in this dev container (no GPU/torch needed for the glue):** unit test that
  `compare()` is deterministic on synthetic candidate rows; a **no-regression invariant test** asserting
  "all new-cue flags OFF ? predictions byte-identical to control"; a CI-gate test asserting
  `run_regression.py` exit codes (0 ok / 1 regress / 3 no-baseline).
- **Machine-side (owner, GPU/WSL):** persist candidates once on a real clip, run `compare(control,
  fusion)` LOVO, confirm the reported lift matches a manual `metrics.score()` spot-check.
- **Docs (P0):** links resolve; indexed in `docs/README.md`; cross-linked from
  `MULTI_SIGNAL_FUSION_PLAN.md` and `GOLDEN_SET_VENDORING.md`.
</content>

```

9. user (2026-06-14T06:50:40.902Z)

```
# A signal-fusion framework for rally detection ? analyzers, scopes & reconcilers

**Status:** **DESIGN ? owner-gated, NOT started.** A peer to
[EXPERIMENT_HARNESS.md](EXPERIMENT_HARNESS.md) and [MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md).
This doc specs a small **framework** for evolving rally-detection quality: producers that emit **signals**
at three temporal **scopes** (frame / window / session), a **scorer** that *learns to weight* them, and an
auditable **reconciliation** pass that lints the decision against the evidence. The goal is to keep piling
on clever cues **without the pipeline becoming a pile of special cases.** Docs-only; no implementation
code. Each phase below is a separate, owner-gated PR.

> **Companion docs:** the detection-layer producers (shuttle / person / audio / shuttle-state) live in
> [MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md) (`RallyCue` = swappable producer; the
> `_enrich_candidate_stats` hook; scale/translation-invariant feature discipline, §3.2); the no-regression
> A/B + LOVO measurement these ride is [EXPERIMENT_HARNESS.md](EXPERIMENT_HARNESS.md) (`Variant`, `compare`,
> `compare_unlabeled`); the scorer is the distilled logistic regression (**ADR-004**,
> `backend/eval/classifier.py`); the modular rally layer is **ADR-007**
> ([archives/decisions/DECISIONS.md](archives/decisions/DECISIONS.md)); the 2-layer mental model is
> [HOW_RALLY_DETECTION_WORKS.md](HOW_RALLY_DETECTION_WORKS.md); the live quality log is
> [QUALITY_ITERATIONS.md](QUALITY_ITERATIONS.md).

> **Framing.** Rally detection is a genuinely hard temporal problem ? weak, multi-modal cues; interval-level
> ground truth that is scarce and expensive; a confound (warm-up / knock-ups / multi-court / held-shuttle)
> that looks identical to real play in any single frame or frequency band. This doc is written as a
> **reusable framework**, not a badminton-specific patch: the contracts are sport-agnostic and the design
> is meant to generalize to other weakly-supervised temporal-segmentation problems (§10). Where that
> ambition touches research claims, §10 says plainly what is and isn't yet
```

substantiated.

---

1. The spine ? NO special-casing in the decision path

This is the one principle everything else serves:

> **Every producer (analyzer, detector, reconciler) emits a SIGNAL carrying a value + a confidence/presence.
> The SCORING MODEL consumes those signals and *learns to weight them*. We do NOT add hand-coded `if/else`
> branches into the decision.**

Producers propose; the scorer arbitrates; the experiment harness + golden set decide what gets promoted.
That separation is what lets us add a net-hit service fault, a let-cord, a double-bounce, a shuttle-out
cue, a *warm-up span*, a *shuttle-state* ? each as *"register a producer + a feature column,"* never *"add
a branch."* A signal that doesn't earn its keep stays at weight ? 0; it costs nothing because it is an
unused column, not a live `if`.

The pattern already exists in miniature: `experiment.predict()` (`backend/eval/experiment.py:203`) treats
`net_crossings` and `players_in_court` as **feature columns** ? `_gate_mask` is the trivial scorer
(threshold a column), `LogisticRegression` (ADR-004) the richer learned scorer (weight all columns), a
future scorer richer still. The decision path is *signals ? scorer*, with **zero** `if service_fault:
drop`. Analyzers feed it more columns; reconcilers run *outside* it as an audited post-pass.

```
 DETECTION ? frame-scope producers (expensive, GPU/WSL, run ONCE)
   WASB shuttle (x,y,v,vis) · person boxes+tracks · audio hits · shuttle-state(in_air/contact/inactive)*
                                          ?  per-frame-aligned signals (+ net estimate, window bounds)
                                          ?
                       shuttle-seeded candidate windows   (FusionSegmenter, P2)
                                          ?
 ANALYZERS ? decision-layer producers (pure, offline, re-scorable)            [forward seam]
   ?? window-scope  (WindowAnalyzer):  one candidate ? features + events + boundary hints   e.g. service-fault
   ?? session-scope (SessionAnalyzer): whole timeline ? labeled spans + per-window membership  e.g. warm-up
                                          ?  every signal ? candidate_segments.stats as feature columns
                                          ?
               SCORING / DECISION  ?  signals ? learned scorer  (weights, NO branches)
               _gate_mask (trivial)  +  LogisticRegression (ADR-004)  +  harness-pinned model
                                          ?  predicted rally segments
                                          ?
 RECONCILERS ? backward seam, "linter for rally decisions" (report-first, idempotent)
   ConsistencyRule registry: decision-vs-evidence conflicts ? Issue + corrected set
                                          ?
   stats · events · spans · corrections record · highlights      EXPERIMENT HARNESS gates EVERY promotion
   (* shuttle-state = a richer frame detector; its PRODUCER lives in MULTI_SIGNAL_FUSION_PLAN, consumed here ? §3.5)
```

---

2. The framework ? signals, scopes, and who arbitrates

The architecture has exactly three kinds of moving part. Capability is added by registering a producer;
the decision logic never grows a branch.

Producers emit signals at three temporal SCOPES. A signal at a coarser scope is computed by aggregating
finer-scope signals; everything ultimately lands as a per-candidate feature column (or an event/span the
scorer or a human consumes).

Scope	Producer	Sees	Emits	Cost	Examples
frame	detector (`RallyCue`)	raw frames / audio	per-frame signal	**expensive** (GPU/WSL)	WASB shuttle, person boxes, audio hits, **shuttle-state** (§3.5)
window	`WindowAnalyzer` (§3.1)	one candidate window's aligned frame-signals	features (+conf), events, boundary hints	cheap, pure	service-fault, double-bounce, let-cord, shuttle-out
session	`SessionAnalyzer` (§3.4)	the whole timeline of windows	labeled spans + per-window membership features	cheap, pure	**warm-up / practice**, game/break phases

The scorer arbitrates. It is the only place a decision is made, and it makes it by *weighting columns*,
never branching. Today: `_gate_mask` (threshold) + `LogisticRegression` (learned weights, ADR-004). The
confidence column on every signal lets the scorer discount an uncertain producer instead of trusting a
binary verdict ? the small-N-safe way to fold in a deterministic-ish cue.

Reconcilers audit. After the scorer decides, the backward seam (§6) lints the decision against the
persisted evidence and proposes corrections ? report-first, never silent.

Four invariants make this safe to grow (the load-bearing design claims):
1. **Signal, not verdict** ? every producer's output is a weighted feature; the decision path has no
   producer-specific branch. ? unbounded cues without combinatorial special-casing.
2. **Expensive detection ? cheap decision** ? frame-scope detection runs once and is persisted; window/
   session signals and the scorer are pure re-scoring (EXPERIMENT_HARNESS §2). ? experiments are offline,
   deterministic, zero production risk.
3. **Report-first reconciliation** ? corrections are recorded and reviewable, never applied silently. ?
   human-in-the-loop, auditable.
4. **Promote only on measured held-out lift** ? LOVO on the golden set + the `run_regression.py` gate;
   default OFF; rollback = config flip. ? no-regression evolution.

---

3. The forward seam ? injectable analyzers

Pluggable producers ? registered like `segmenter_registry` (`backend/pipeline/segmenters/base.py:35`) ?
that run over the **per-frame-aligned signals already produced**: the WASB shuttle trajectory
(`.frame/.visible/.x/.y` + velocity), the person boxes per frame (`PersonDetector.detections_per_frame`,
`person_detector.py:256`), the net line (`rally_gate.estimate_play_axis`, camera-agnostic), and the window

bounds. Analyzers are **pure** ? no I/O, no torch/cv2 ? mirroring `fusion_features.py`'s discipline, so
their math is unit-testable in the GPU-free dev container.

3.1 Contracts (JSON-serializable; proposed `backend/pipeline/analyzers/`)

```python
@dataclass(frozen=True)
class Signal:                           # one scalar cue + how sure the producer is ? never a verdict
    value: float; confidence: float = 1.0    # persisted as `<producer>__<key>` (+ `_confidence`), [0,1]

@dataclass(frozen=True)
class AnalyzerEvent:                     # ? the `events` table on confirmation (§5)
    type: str; timestamp: float; confidence: float = 1.0   # "net_hit"/"double_bounce"/?; src-absolute s
    player: Optional[str] = None; details: Dict[str, Any] = field(default_factory=dict)

@dataclass(frozen=True)
class BoundaryHint:                      # OPTIONAL deterministic proposal ? never auto-applied
    kind: Literal["start", "end"]; timestamp: float; confidence: float; reason: str

@dataclass(frozen=True)
class AnalyzerResult:
    signals: Dict[str, Signal] = field(default_factory=dict)
    events: List[AnalyzerEvent] = field(default_factory=list)
    boundary_hints: List[BoundaryHint] = field(default_factory=list)

@dataclass
class WindowSignals:                     # read-only pre-produced bundle (no I/O inside the analyzer)
    start: float; end: float; fps: float; frame_width: float; frame_height: float
    shuttle: List["TrajectoryPoint"]     # in-window points (.frame/.visible/.x/.y)
    persons: Dict[int, List[Tuple[int, BBox]]]  # frame_idx ? [(track_id, box)]; {} when person cue OFF
    net: Tuple[str, float]               # (axis, position): rally_gate auto-estimate or config
    frame_states: Dict[int, "ShuttleState"]      # frame_idx ? state; {} until the shuttle-state detector (§3.5)
    base_stats: Dict[str, float]         # motion stats already computed for the window

class WindowAnalyzer(ABC):
    name: str; version: str              # registry key + column prefix; version ? provenance on change
    feature_keys: List[str]; event_types: List[str]    # declared columns/events (for the harness)
    @abstractmethod
    def analyze(self, w: WindowSignals) -> AnalyzerResult: ...
```

The three roles an `AnalyzerResult` can fill:

Role	Sink	Consumed by
(a) features (with confidence)	`candidate_segments.stats` as `<analyzer>__<key>` (+ `_confidence`)	the scoring model **and** the harness, as feature columns (§4)
(b) events	the `events` table (buffered on the candidate, flushed on confirmation, §5)	history / highlights / audit
(c) boundary hints (optional)	stashed in `stats`; never auto-applied	the reconciler (§6) / a future boundary-aware scorer

3.2 Worked example ? the SERVICE-FAULT analyzer (window-scope)

The owner's held-shuttle bug has a sibling: a **service fault** where the serve hits the net and the shuttle comes to rest in the net region. Pure trajectory, GPU-free, no person cue needed:

- **Input:** the window's shuttle trajectory + the `rally_gate` net estimate.
- **Logic:** the shuttle **enters the net region** (`|coord ? net| < deadband`) **and** its velocity **decays to ? 0** there (comes to rest) ? high-probability net-hit service fault. Confidence scales with how cleanly the halt sits inside the net band.
- **Emits:** a `service_fault` **signal** (value = presence, confidence = halt cleanliness) + a `net_hit` **event** at the halt timestamp + a rally-**END** `BoundaryHint` at that timestamp.

Crucially: the analyzer does **not** drop the window. It *proposes* `service_fault`; the scorer sees `service_fault` + `service_fault_confidence` as two more columns and **learns** whether/how much to down-weight. The boundary hint is an audited proposal the reconciler may act on ? never a branch.

3.3 Injection point ? generalize `_enrich_candidate_stats`

The FusionSegmenter hook (`fusion_hybrid.py:70`) already turns "compute fixed features" into per-window enrichment. We generalize it to **"run each enabled analyzer"** ? additively, so the existing `net_crossings`/person paths are unchanged:

```python
def _enrich_candidate_stats(self, cw, points, fps, frame_width, video_path, idx_cfg):
    stats = super()._enrich_candidate_stats(...)        # base motion keys
    stats["net_crossings"] = ...                        # Gate-A signal (kept)
    # ? person features when the person cue is on (kept) ?
    w = self._window_signals(cw, points, fps, frame_width, video_path, idx_cfg)
    for analyzer in analyzer_registry.enabled(idx_cfg):  # NONE enabled ? byte-identical to control
        res = analyzer.analyze(w)
        for key, sig in res.signals.items():
            stats[f"{analyzer.name}__{key}"] = sig.value
            stats[f"{analyzer.name}__{key}_confidence"] = sig.confidence
        stats.setdefault("_events", []).extend(e.__dict__ for e in res.events)        # flushed in Phase 4
        stats.setdefault("_boundary_hints", []).extend(h.__dict__ for h in res.boundary_hints)
    return stats
```

Flag-gated, **default OFF** via the existing `indexing.fusion.cue_flags` (a sibling `analyzer_flags` map keyed by `analyzer.name`). With every analyzer OFF, the loop is empty ? candidate rows are byte-identical to the substrate ? the no-regression invariant holds (§7). `net_crossings` and the person features are the **prototype analyzers** the seam retrofits first (proving it with zero behavior change); every new cue after that is a pure add.

3.4 Session-scope analyzers ? the WARM-UP / PRACTICE example

Some structure is invisible at the window scope. **Warm-up** looks exactly like a rally in any single window ? shuttle in flight, two players, net crossings, sustained motion ? which is *why* it is a notorious

false-positive source. It is only distinguishable by **timeline structure**: a long contiguous active
stretch *without* the rally?dead-time?rally cadence of scored play, usually (not always) at the start.

So warm-up needs a producer that sees the **whole sequence of windows**, not one window. That is a sibling
tier ? a `SessionAnalyzer` that runs once per video (a second pass, after per-window enrichment) and emits
a labeled **span** plus a per-window **membership feature**:

```python
@dataclass(frozen=True)
class Span:
    kind: str                           # "warmup" | "practice" | "break" | "game" | ?
    start: float; end: float; confidence: float; reason: str

@dataclass(frozen=True)
class SessionResult:
    spans: List[Span] = field(default_factory=list)
    # window_index ? signals merged back into THAT window's stats (e.g. {"in_warmup": Signal(0.8, 0.6)})
    window_features: Dict[int, Dict[str, Signal]] = field(default_factory=dict)

class SessionAnalyzer(ABC):
    name: str; version: str
    span_kinds: List[str]; feature_keys: List[str]
    @abstractmethod
    def analyze(self, windows: List[WindowSignals], session: "SessionContext") -> SessionResult: ...
```

`WarmupAnalyzer` (worked):
- **v0 (free, no new model):** find the **leading contiguous high-activity span** with low inter-window
  dead-time and no serve-reset cadence, over the persisted window timings we already have. Emit
  `Span(kind="warmup", ?)` + per-window `in_warmup` = overlap fraction of the window with the span.
- **v1 (learned):** fed by the **shuttle-state cadence** (§3.5) / the **audio thwack** cadence ? warm-up =
  continuous hits *without* the point-ending pause rhythm; generalizes to mid-session practice / between-
  game knock-ups (drop the position prior).

You framed the consumption exactly right: "so the scorer may consume it." `in_warmup` is a **feature
column**; the scorer *learns* to down-weight windows inside the span. **Warm-up is the poster child for
no-special-casing**: the naive version is `if t < warmup_end: drop` ? a hand-coded branch hanging off a
fuzzy boundary; a 20-second error **deletes a real opening rally**. As a weighted feature, a wrong boundary
just slightly mis-weights a couple of windows the other cues rescue. The span is *also* persisted as
evidence a reconciler (§6) may use to down-weight a whole cluster, but the primary path is feature ? scorer.

It is measurable today with no new labels: the golden labels already treat warm-up knock-ups / pre-serve
as **dead time = negatives** (Roora starts a rally at serve-contact; fusion §1 names "warm-up knock-ups" as
a precision target). So an `in_warmup` feature is directly A/B-able on the 6 golden videos for **precision**
lift ? that is the promotion gate.

3.5 Consuming a richer detector ? SHUTTLE-STATE (frame-scope producer)

A natural new **detector**: a per-frame shuttle-state classifier ? `in_air` / `racket_contact` /
`inactive`. Note the scope: it **processes frames ? a new per-frame signal**, like WASB, so it is a
frame-scope producer in the detection layer ? its spec belongs in
MULTI_SIGNAL_FUSION_PLAN.md (it is a new `RallyCue`), **not** a `WindowAnalyzer`.
This doc records **how the framework consumes it** once it exists:

- a **`WindowAnalyzer`** turns the persisted per-frame state into window features ? `racket_contact_count`
  (? shot cadence), `inactive_ratio` (? dead-time) ? that flow into the scorer as columns;
- a **`SessionAnalyzer`** (warm-up, §3.4) reads its cadence;
- the **reconcilers** (§6) get *stronger* evidence: `trailing_dead_time` / `over_merged` currently infer
  inactivity from a velocity threshold; a learned `inactive` state is a more direct, occlusion-robust
  evidence source for "truncate / split here."

Stage it (don't build the model first). Most of the state is derivable for free from signals we already
have ? `in_air` vs `inactive` ? WASB velocity + visibility; `racket_contact` ? trajectory **direction
reversals** (fusion §4: "each reversal ? a shot") **and** the scaffolded **audio thwack**
(`backend/pipeline/audio/hit_detector.py`, ADR-007's #1 cue). So **v0 = a derived `shuttle_state` analyzer**
over existing signals (no new model); measure it. **The dedicated per-frame vision model earns its keep
only where v0 hits a ceiling: WASB trajectory gaps** ? occlusion, motion blur, fast smashes ? where there
is no `(x,y)` to threshold but an appearance model could still call `racket_contact`. **Label blocker:** we
have **zero** per-frame state labels (golden labels are rally intervals); a dedicated model needs weak/
distilled labels or a new (minors-gated) annotation task ? the same gate as feature-level fusion. If you can
weak-label state from existing signals, prefer the derived analyzer until the gap-filling case is proven.

---

4. How signals reach the scorer ? columns, not branches

The persisted `stats` columns are exactly what the harness already lifts. `load_video_candidates(...,
feature_names=[...])` (`experiment.py:429`) reads each name out of the row's `stats` JSON into a feature
column; `predict()` (`experiment.py:203`) gates/scores on those columns and merges. So a new producer's
signal becomes a scorer input by **adding its column name to a `Variant`'s `feature_names`** ? no new
scoring code, no branch:

```python
# A learned Variant that lets the scorer weight the new signals (service-fault + warm-up membership):
fusion_plus = Variant(name="fusion+analyzers",
                      feature_names=[*SHUTTLE_FEATURES, "person__players_in_court",
                                     "service_fault__service_fault", "service_fault__service_fault_confidence",
```

```
                                        "warmup__in_warmup", "warmup__in_warmup_confidence"])
compare(golden_videos, CONTROL, fusion_plus)        # LOVO-honest B ? A on the 6 golden videos
```

The model **learns the weights** ? down-weight a window with a high-confidence `service_fault` or high `in_warmup`; up-weight ?2 in-court players + multiple net-crossings. **What the scorer cannot see, it cannot special-case; what it can see, it weights from data.** Same logistic regression as ADR-004; we add columns, never a net.

---

5. Persistence

- **Features ? `candidate_segments.stats`** (`database.py:240`). Namespaced `<producer>__<key>` (+ `_confidence`). Persisted **as a shadow** even when the served decision ignores them, so any future `Variant` re-scores offline (EXPERIMENT_HARNESS §5.2). `version`/`name` recorded for provenance.
- **Spans ? window membership + a session record.** A `SessionAnalyzer`'s per-window `in_<span>` features ride in each window's `stats` (so they re-score like any feature); the span list itself (start/end/kind/confidence) is a small per-video record (a JSON sidecar first; a table only if it grows).
- **Events ? the `events` table** (`database.py:227`, `add_event(segment_id, ts, type, player, details)`). Its FK is to `play_segments`, which exist only **after** the AI handoff confirms a candidate. So analyzer events are **buffered on the candidate** (`stats["_events"]`) and **flushed** to `events` in Phase 4, when `_candidate_id ? segment_id` is linked (`trajectory_hybrid.py:283`). Candidate-level discoveries survive for offline re-scoring; the `events` table stays "events on confirmed segments."
- **Corrections record** (§6) ? report-first means corrections are **recorded, never silent.** Phase one: a JSON sidecar shaped like `experiment_leaderboard.json` (one row per issue: segment, kind, evidence, before?after, reason, timestamp). When auto-apply is promoted, add a `corrections` table via the `PRAGMA user_version` migration ladder (`database.py:88`, `if version < 3:`) so applied corrections are queryable and reversible.

---

6. The backward seam ? the reconciliation pass (a linter for rally decisions)

Given **predicted segments + the persisted per-frame/per-window/per-session signals**, detect where the **DECISION contradicts the EVIDENCE** and propose corrections. This is the backward dual of the analyzers: analyzers add evidence *before* the decision; reconcilers audit the decision *against* evidence *after* it.

6.1 Contracts (proposed `backend/pipeline/reconcilers/`)

```python
@dataclass(frozen=True)
```

```
class Issue:
    segment_index: int; kind: str       # kind = rule id, e.g. "trailing_dead_time"
    evidence: Dict[str, Any]            # the signal values that triggered it (auditable)
    before: Interval; after: Optional[List[Interval]]   # None=drop · [iv]=corrected · [iv1,iv2]=split
    reason: str; confidence: float

@dataclass(frozen=True)
class Correction:
    issues: List[Issue]; corrected: List[Interval]      # the FULL corrected set (idempotent)

class ConsistencyRule(ABC):
    name: str; precedence: int          # FIXED ordering; lower runs first
    @abstractmethod
    def check(self, seg: Interval, signals: "SegmentSignals") -> Optional[Issue]: ...

def reconcile(segments, signals, rules) -> Correction: ...   # fixed precedence; idempotent
```

6.2 Worked rules (fixed precedence)

#	Rule	Evidence contradiction	Correction
1	`trailing_dead_time`	segment still "in rally" at its end, but the shuttle went inactive earlier	**truncate** to the last-active frame *(the owner's misfire fix)*
2	`no_net_crossing`	`net_crossings < 1` ? nothing ever crossed the net	**drop** (or flag)
3	`over_merged`	a dead-time gap **>** `merge_gap` of shuttle-inactivity sits **inside** one segment	**split** at the gap
4	`boundary_slack`	first sustained shuttle activity / first net-crossing is well inside the start	**tighten** start to serve-contact
5	`inside_warmup`	the segment lies inside a high-confidence `warmup`/`practice` span (§3.4)	**flag** (drop only under apply-flag)

Each rule reads only **persisted evidence** (trajectory activity per frame, `net_crossings`, person occupancy, analyzer signals/events, session spans, shuttle-state) ? it is a deterministic linter, not a re-detector. As richer producers land (shuttle-state, warm-up), rules get **stronger evidence**, not new branches in the decision path.

6.3 Report-first, apply-optional (non-negotiable)

`reconcile()` returns **both** an issues report **and** a corrected set. It **NEVER silently rewrites** (auditability + the repo's honesty/attribution working agreement). The pipeline either:

- **(default)** persists the report and **surfaces it for human review** ? production output unchanged; or
- **(flagged, `reconcile.apply`)** applies the corrected set, **recording every change** in the corrections record (before?after, rule, evidence). Rollback = flip the flag.

It is a `Variant` transformation, so its lift is A/B-measurable BEFORE auto-apply. A reconciler is a pure post-`predict` stage: `reconcile(predict(...), signals, rules).corrected`. Wrapping that as a `RECONCILED` variant lets `compare(CONTROL, RECONCILED)` report the LOVO lift on the 6 golden videos (QUALITY_ITERATIONS.md corpus) ? promote to auto-apply **only** on measured lift.

Idempotent; fixed rule precedence. `reconcile(reconcile(x)) == reconcile(x)` (truncating an already-truncated segment is a no-op); precedence is a fixed table so the result is deterministic and order-independent of input segment order.

---

7. No-regression discipline (identical to the harness contract)

The same four guarantees EXPERIMENT_HARNESS.md §6 makes ? applied to every producer and the reconciler:

- **Default OFF.** Analyzers gate on `analyzer_flags` (default empty); the reconciler is report-first (apply gate default off). Merged code **cannot** change served behavior until explicitly enabled.
- **Shadow-persist, then promote.** Signals/spans/issues are persisted even while ignored, so a `Variant` re-scores them offline; a signal becomes a *decision input* only when a Variant that references it wins LOVO. (EXPERIMENT_HARNESS §5.2's shadow mode.)
- **Promotion only via the eval gate.** `run_regression.py` against the golden `baseline.json` (exit 1 blocks); the experiment leaderboard records the spec hash + per-video + LOVO deltas + label-set hash.
- **Rollback = flip a config flag, never `git revert`.** The pinned `CONTROL` variant is always present.

The keystone test (extends the harness's existing invariant): **all analyzer flags OFF + reconciler report-only ? candidate rows AND predictions byte-identical to control.**

---

8. How it composes with fusion P2

- **Window-analyzers plug into fusion P2's enrichment hook.** `FusionSegmenter._enrich_candidate_stats` (`fusion_hybrid.py:70`) is the exact injection point §3.3 generalizes; `net_crossings` and the person features become the first registered analyzers (no behavior change). New cues are add-a-producer, per ADR-007 / MULTI_SIGNAL_FUSION_PLAN §2.
- **Session-analyzers run as a second pass** over the per-window stats the same hook persists.
- **The shuttle-state detector is a new fusion `RallyCue`** (frame-scope producer) ? it belongs in the fusion plan; this framework only *consumes* it (§3.5).
- **Reconcilers consume fusion P2's persisted signals** ? the same `candidate_segments.stats` plus the persisted trajectory; no new detection.
- Everything inherits fusion P2's control-reproducing defaults, so the fusion segmenter with all flags off still seeds and hands off identically to `wasb_hybrid`.

---

9. Trade-offs & risks

- **Deterministic vs learned signals.** A producer may emit a verdict-shaped signal (`service_fault=1.0`), but we persist it as a **feature**, so the scorer still arbitrates. Risk: a confidently-wrong cue.

Mitigation: the **confidence column** + the **LOVO gate** ? a bad signal earns ? 0 weight, never promotes.
- **The temporal confound is the hard core.** Warm-up-vs-play, multi-court, and held-shuttle all look identical in any single frame / frequency band ? QUALITY_ITERATIONS' standing lesson: "the dead-time confusion is rally-vs-non-rally activity in the same place and frequency band... only the learned model's temporal features crack it." This is *why* warm-up is a **feature the model weights**, not a heuristic span-cutter, and why session-scope structure earns its own tier.
- **Scope discipline.** Put a producer at its *natural* scope: shuttle-state is frame-scope (a detector), not a window analyzer; warm-up is session-scope, not per-window. Mis-scoping forces a producer to either recompute detection or miss the structure it needs.
- **Ordering / idempotence (reconcilers).** Fixed precedence table + an idempotence test (`reconcile?reconcile == reconcile`).
- **Signal, not verdict.** A producer that *decides* (returns keep/drop) re-introduces special-casing; reviews must reject "analyzer returns a boolean keep/drop."
- **Report-first auditability.** Reconcilers never silently mutate output; every applied change is recorded.
- **Small-N feature discipline.** Few, scale/translation-invariant columns (counts/ratios/presence, never raw coords ? MULTI_SIGNAL_FUSION_PLAN §3.2); the model stays a logistic regression, not a net.
- **Labels gate the learned producers.** Shuttle-state and feature-level fusion need labels we don't yet have; warm-up is measurable *now* because golden treats it as negatives. Know which producers are gated.
- **Cost & licensing.** Pure analyzers are GPU-free; person- and shuttle-state-dependent producers pull torch / a second WSL pass ? gate behind their cue flags; all deps stay MIT/Apache/BSD (ADR-002/005).
- **Event-table timing.** The buffer-then-flush wiring (§5) is the one non-obvious integration detail.

---

10. Generality & research positioning

This is written as a **framework**, not a badminton patch. Read sport-agnostically it is: *a registry-based, multi-scope **signal-fusion** architecture for **weakly-supervised temporal event/segment detection**, with a learned arbiter and an **auditable post-hoc correction** layer.* The pieces that generalize:

- **Multi-scope producers ? one feature table ? a learned arbiter.** The frame/window/session taxonomy and the "signal-not-verdict" rule are domain-independent. Any temporal problem with **weak, multi-modal cues at different time scales** (broadcast sports event spotting, surgical-phase recognition, call-center / meeting segmentation, activity logs) fits the same shape: add a producer, add a column, let the arbiter weight it, gate promotion on held-out lift.
- **Detection ? decision (persist once, re-score forever).** Decouples the expensive, hard-to-reproduce perception from the cheap, deterministic decision ? the property that makes the whole thing experimentally honest and cheap to iterate.
- **Auditable reconciliation as a first-class layer.** Most pipelines bury post-hoc fixes as silent

if-branches; treating consistency-vs-evidence as a *report-first, A/B-measured* pass is the transferable methodological idea (and the human-in-the-loop / safety story).
- **The honest measuring stick.** Interval-level, scarce ground truth + **leave-one-VIDEO-out** (group by match, never a held-out window from the same video) is the discipline any small-corpus temporal study needs to avoid flattering itself.

What is defensibly contributed vs. what is standard. Multi-modal **late fusion**, weak supervision, and learning a linear/logistic arbiter over hand-designed features are all standard. The *combination* offered here ? (a) a uniform producer-registry across **three temporal scopes**, (b) a hard **no-branch** decision invariant that makes extension monotone, (c) a **report-first, harness-measured reconciliation** layer, and (d) the **detection?decision persistence** substrate that makes every experiment a zero-risk offline re-score ? is what would be written up, with rally detection as the motivating hard problem.

Honesty caveat (attribution working agreement). No external results or citations are asserted here. A real write-up still owes: a proper **related-work survey** (temporal action segmentation/detection, sports event spotting, multimodal fusion, weak supervision ? to confirm which claims are actually novel), **strong baselines**, **per-producer ablations**, and reporting on **>1 sport / camera** to substantiate the generality claim. Today the corpus is **6 matches** (QUALITY_ITERATIONS) ? enough for direction-checks and LOVO sign-tests, **not** enough to claim a general model; corpus growth is the highest-ROI unblock. Treat §10 as a *positioning sketch*, not a results section.

> **Update (2026-06-14): that related-work survey is now done ? see [§13](#13-prior-art-terminology--course-corrections-literature-review-2026-06-14).**
> The honest finding: the *skeleton* (TAL/TAS + late fusion) is standard and must be **cited, not claimed**;
> the only `novel-combination` is the report-first reconciler-as-linter bundle; warm-up-as-a-soft-feature is
> the thinnest genuine gap. §13 also lists the verified citations and the prioritized course corrections.

---

11. Phased execution (owner-gated; ONE PR per slice, off `master`, default OFF)

- **A0 ? this doc**, indexed in `docs/README.md`, cross-linked from the peers + ADR-007. *(this deliverable; docs-only.)*
- **A1 ? analyzer contracts + registry** (`WindowAnalyzer`/`AnalyzerResult`/`Signal`/`AnalyzerEvent`/ `BoundaryHint` + `analyzer_registry`); generalize `_enrich_candidate_stats` to iterate enabled analyzers (none enabled ? byte-identical). Retrofit `net_crossings` + person features as the first two analyzers.
- **A2 ? service-fault analyzer** behind `analyzer_flags.service_fault` (default OFF); signal + `net_hit` event + end boundary hint; shadow-persisted; offline `compare(CONTROL, +service_fault)` LOVO.
- **A3 ? event flush wiring** (buffer analyzer events on candidates ? `add_event` on confirmation).
- **A4 ? `SessionAnalyzer` tier + `WarmupAnalyzer` v0** (leading-span heuristic over

window timings), default OFF; persists `warmup__in_warmup` + the span record; measured for **precision** lift on golden.
- **A5 ? derived `shuttle_state` analyzer v0** (velocity + visibility + direction-reversals (+ audio); **no new model**); emits `racket_contact_count` / `inactive_ratio`; measured LOVO. *(The dedicated per-frame shuttle-state DETECTOR is a separate MULTI_SIGNAL_FUSION_PLAN `RallyCue` item ? gated on per-frame labels; build only if v0 is blocked by trajectory gaps.)*
- **R1 ? reconciler contracts + registry + `reconcile()`** (`ConsistencyRule`/`Issue`/`Correction`) with the worked rules; pure, idempotent, unit-tested; **not** wired to apply (report-only).
- **R2 ? reconciler as a `Variant` transform**: optional post-`predict` stage; `compare(CONTROL, RECONCILED)` LOVO; report-only in production.
- **R3 ? corrections persistence + opt-in apply**: JSON sidecar first, then the `corrections` table (`user_version < 3`); apply behind `reconcile.apply`; **rollback = flip the flag.**

Each PR is gated by the no-regression invariant (§7) + `run_regression.py`. Order is a suggestion; A4/A5/R1 are independent and can be reprioritized on owner steer.

---

12. Verification

- **Offline, runnable in this dev container (no GPU/torch/cv2):** pure analyzer math on synthetic trajectories + person boxes (service-fault fires on a synthetic net-halt, silent on a clean rally; `WarmupAnalyzer` v0 finds a synthetic leading knock-up span; derived `shuttle_state` matches a scripted velocity/visibility sequence); the **no-regression invariant** (all flags OFF + reconciler report-only ? candidate rows + predictions byte-identical to control); the **reconciler idempotence** test (`reconcile?reconcile == reconcile`) and fixed-precedence determinism; `compare(CONTROL, variant)` deterministic on synthetic candidate rows.
- **Metrics ? adopt the field's (see [§13](#13-prior-art-terminology--course-corrections-literature-review-2026-06-14)/C4):** report **segmental F1@{10,25,50} + edit score** (TAS) and **tight/loose mAP@?** (action spotting) alongside temporal-IoU. tIoU alone is nearly blind to the over-segmentation the reconciler exists to fix, so a reconciler win ? or an IoU-chasing rule that shreds rally structure ? is invisible under tIoU.
- **Machine-side (owner, GPU/WSL ? full pipeline can't run here):** persist analyzer signals on a real clip; `compare(CONTROL, +service_fault)`, `compare(CONTROL, +warmup)`, `compare(CONTROL, RECONCILED)` LOVO on the 6 golden videos; confirm the owner's **trailing-dead-time misfire** truncates, the **held-shuttle / net-hit service feed** is down-weighted, and **warm-up windows** lose precision-mass; spot-check `net_hit` events on confirmed segments. **Grade with §13/C4 metrics + per-fold CIs (C1).**
- **Docs (A0):** links resolve; indexed in `docs/README.md`; cross-linked from MULTI_SIGNAL_FUSION_PLAN.md, EXPERIMENT_HARNESS.md, HOW_RALLY_DETECTION_WORKS.md, and ADR-007.

---

13. Prior art, terminology & course corrections (literature review, 2026-06-14)

A multi-agent web-research pass surveyed the eight design patterns of this framework against the literature and **adversarially verified every citation** (real-paper check + closer-prior-art hunt). **Result: zero fabricated papers**; a few wrong author-lines / metadata slips were caught and are corrected below. This section is the honest "are we reinventing the wheel?" answer and the actionable course corrections ? all **owner-gated, NOT yet implemented** (they fold into the A*/R* phases of §11).

13.1 Verdict ? what is standard vs. what is ours

> **TL;DR.** The architecture's *skeleton* is standard and must be **cited, not claimed**: candidate-windows + learned scorer = **Temporal Action Localization** (proposal-then-score); the reconciler's over-segmentation fixes = **Temporal Action Segmentation** refinement/post-processing; "signal not verdict / learned arbiter over cue columns" = **late / score-level fusion** realized as **stacked generalization** ? ~20 years old and *already present in racket sports*. The **defensible delta is the engineering/governance envelope**: detection?decision persisted as a re-score substrate + a **report-first, A/B-lift-gated, idempotent, human-in-the-loop reconciler-as-linter** keyed on decision-vs-evidence *contradiction* + the flag-gated no-regression promotion harness, on a deliberately tiny LOVO-by-match golden set with an interpretable scorer. Of the patterns, only the **reconciler-as-linter** rated `novel-combination`; the rest are `partial-overlap`. Park any novelty claim narrowly on (a) that reconciler bundle and (b) **warm-up-as-a-soft-session-feature** (the one slice with genuinely thin prior art).

Pattern (our framing)	Field's name	Closest verified prior art	Verdict
Candidate-windows + scorer + reconciler-as-refinement	Temporal Action Localization / Segmentation + boundary refinement	ASRF (arXiv:2007.06866); Activity Grammars (arXiv:2312.04266); BMN (arXiv:1907.09702)	partial-overlap
"Signal not verdict" ? learned arbiter over cue columns	Late / score-level fusion + stacked generalization	Snoek 2005 (10.1145/1101149.1101236); Wolpert 1992; racket-sports A/V rally-break fusion (CVIU 2009)	partial-overlap
Detection?decision, "persist once, re-score forever"	Frozen-backbone cached features + decoupled proposal/classification	S-CNN (arXiv:1601.02129); CBR (arXiv:1705.01180); Decouple-SSAD (arXiv:1904.07442)	partial-overlap (? field default)
Report-first reconciler (contradiction ? audited correction, measured before apply)	Model assertions + constraint-based repair ("ML linter")	Model Assertions (arXiv:2003.01668); HoloClean (arXiv:1702.00820); TFDV (MLSys 2019); EventAnchor (arXiv:2101.04954)	**novel-combination**
Scarce interval labels + tiny LR + LOVO-by-match	Point/timestamp-supervised TAL + grouped (LOGO/LOSO) CV	SF-Net (arXiv:2003.06845); LOOCV-bias (arXiv:2406.01652); Snorkel (arXiv:1711.10160)	partial-overlap
Multi-scope (frame/window/session); warm-up conditions per-window	Multi-scale/hierarchical modeling + play-break + phase recognition	Tjondronegoro ICME 2003; GL-MSTCN (10.1186/s12938-022-01048-w); MS-TCN (arXiv:1903.01945)	partial-overlap

In-domain prior art that already hard-codes our cues (so our delta is the *no-branch discipline*, not the cues): non-broadcast badminton rally start/end (See et al. 2024/2025), shuttle-state/hit detection (MonoTrack arXiv:2204.01899; Hsu/Yu/Cheng, Sensors 2024 24(13):4372), service-fault (Goh et al., Sensors 2023 23(24):9759), tennis in-play state machines (10.1080/19346182.2013.819007), badminton point-segmentation (Ghosh et al. 2018, arXiv:1712.08714), ShuttleSet (arXiv:2306.04948).

13.2 Terminology to adopt (gloss our nouns with the field's on first use)

Our term	Field's established term	Anchor
`RallyCue` / "producer"	**labeling function** / weak-supervision source; **base model** in a stack	Snorkel; Wolpert 1992
"Signal not verdict / no special-casing"	**late / score-level fusion**; arbiter = **level-1 combiner / meta-learner**	Snoek 2005; Wolpert 1992
ADR-004 LR scorer (`backend/eval/classifier.py`)	**stacked meta-learner / level-1 combiner**	Wolpert 1992
"Detection?decision / persist once, re-score"	**frozen-backbone cached features**; **decoupled proposal/classification**; offline re-scoring	S-CNN; Decouple-SSAD
`WindowAnalyzer` (window-scope)	**temporal action proposal** features / **actionness**	S-CNN; BMN
Session-scope / warm-up	**video-level temporal context / phase segmentation**; **play-break** outer layer; **global-local**	Tjondronegoro 2003; GL-MSTCN
`ConsistencyRule` reconciler	**model/consistency assertions**; **constraint-based repair**; **boundary refinement**; **activity grammar / constrained decode**	Kang 2020; HoloClean; ASRF; Activity Grammars
"Report-first linter"	**data validation / schema anomalies** ("unit tests for data")	TFDV; Deequ
"Over-merged / split" · "tighten boundary slack"	**over-segmentation error** · **boundary regression/refinement**	MS-TCN; ASRF; CBR
"LOVO, group by match"	**leave-one-group-out / -subject-out (LOGO/LOSO)** grouped CV	sklearn `GroupKFold`; Austin 2025
Temporal-IoU metric	**mAP@tIoU** (TAL) + **F1@{10,25,50} & edit score** (TAS) + **tight average-mAP** (spotting)	Lea 2017; SoccerNet; Soares 2022

Keep the internal nouns; just gloss each on first use. Low-effort, high-leverage ? reviewers map instantly to 30 years of literature and see we know the failure modes.

13.3 What we can / cannot claim

- ? **The integrated governance envelope as a whole** ? no found paper bundles all of: report-first contradiction detection over a *persisted temporal decision* + auditable structured interval corrections (truncate/split/drop/tighten) + **per-rule A/B lift-gating before auto-apply** + idempotency/precedence + human-in-the-loop + feeding a learned (value, confidence) scorer.
- ? **"Signal not verdict" as a deliberate *design-philosophy delta*** vs. the in-domain prior art that hard-codes the same cues as if/else verdicts (state it as a philosophy delta, not a capability they lack).
- ? **Warm-up/practice as a soft session-scope feature** ? thin prior art (only patents/coaching content surfaced); claim cautiously, validate from scratch.
- ? The **fusion mechanism** (multi-cue features ? light scorer for rally segmentation) ? ~20 yrs old in racket sports (CVIU 2009; Ghosh et al. 2018 learn an SVM over per-frame features ? smoothed rallies).
- ? The **persist-once/re-score substrate** as novel ? it is the *default* of the TAD field (datasets ship precomputed features so cheap heads re-train). Claim only the *idempotent/versioned discipline*.
- ? The **cross-check-and-correct insight** ? Hsu et al. 2024 already cross-check shuttle-vs-swing streams; EventAnchor already does human-in-the-loop suggest-then-verify. Our delta is *measured-before-apply +

auditability*, not the cross-check.
- ? **Static single-camera as a novel setting**, and ? the **no-regression/flag-gated harness** as a primitive (textbook MLOps) ? claim only "this discipline on a deliberately tiny golden set."

13.4 Course corrections (owner-gated; prioritized)

Tier 1 ? protect the honesty of "measured lift" on ~6 matches (do first):
- **C1 ? bias-aware LOVO.** Standard leave-one-out on ~6 imbalanced groups induces a train-vs-heldout label anti-correlation that biases the estimate (a LR scored ~0.23 auROC on *random* data in the cited study). Use class-balance-preserving folds (RLOOCV) and **report per-fold variance + CIs, never a single mean**, so the promotion gate can't promote noise. *(Austin/Pe'er/Korem 2025, arXiv:2406.01652.)* ? **harness/eval.**
- **C2 ? calibrate each producer's confidence** (Platt/isotonic) *before* the LR, so a CNN softmax, an audio onset, and an integer net-crossing count are comparable and the over-confident cue can't swamp the arbiter. *(MoCaE arXiv:2309.14976; modality competition arXiv:2203.12221.)* ? **producers + scorer.**
- **C3 ? out-of-fold (honest) stacking.** When a producer is itself learned (shuttle-state), feed the LR its *out-of-fold* outputs; extend the group-by-match split to producer training. *(Wolpert 1992.)* ? **detection?decision + harness.**
- **C4 ? report the field's metrics.** Add **segmental F1@{10,25,50} + edit score** and a **tight/loose mAP@?** sweep alongside temporal-IoU: tIoU is nearly blind to over-segmentation, so a reconciler win (or an IoU-chasing rule that shreds structure) is invisible under tIoU alone. *(MS-TCN; ASRF; SoccerNet arXiv:1804.04527; Soares 2022 arXiv:2206.07846.)* ? **harness/eval + reconciler.**

Tier 2 ? architecture/discipline upgrades:
- **C5 ? resolve reconciler rules GLOBALLY** (an activity-grammar / constraint-aware Viterbi decode), not as stacked local edits with fixed precedence (which can conflict / be non-idempotent across orderings). Model the structural priors (transition/duration) from data. *(Activity Grammars arXiv:2312.04266; Constraint-Aware Decoding arXiv:2605.10149 ? read before citing.)* ? **reconciler.**
- **C6 ? brand the reconciler as "model/consistency assertions,"** attach a per-correction confidence, and route only high-confidence corrections to auto-apply, the rest to a margin-ranked human queue; version the rule set as a TFDV-style owner-ratified anomaly schema. *(Kang 2020; HoloClean ~90% repair precision ? ~10% bad; Confident Learning arXiv:1911.00068.)* ? **reconciler + harness.**
- **C7 ? a few EARLY-fusion interaction columns** (e.g. `shuttle_in_air AND person_two_sided`) ? a linear arbiter can't represent AND/XOR; hand-crossed products recover it without leaving "signal not verdict." *(Snoek 2005.)* ? **producers + scorer.**
- **C8 ? represent a missing cue as an explicit `is_present`/uncertainty column, not a silent 0** ? "cue absent" ? "cue says no" for a linear arbiter. *(DBF arXiv:1511.03183.)* ? **producers + scorer.**
- **C9 ? keep warm-up/session-state a SOFT feature, never a hard pre-filter**, and add a court/context invariance probe: the static court co-occurs with rallies, so the scorer can learn "court visible" instead of "point in play" (action-context confusion). Evaluate on warm-up stretches that

share the court but lack point cadence. *(arXiv:2103.16155; GL-MSTCN.)* ? **session-scope producers + scorer.**

Tier 3 ? leverage external resources / higher-effort:
- **C10 ? model cue CORRELATIONS** (Snorkel/MeTaL label-model baseline; strong L1/L2): net-crossing, two-sidedness, shuttle-in-air co-fire, so an unregularized LR double-counts the bundle. *(arXiv:1711.10160.)*
- **C11 ? a temporal-smoothing pass** (Kalman/HMM/Viterbi over the per-window posterior, or an MS-TCN truncated-MSE anti-fragmentation penalty) before cutting intervals ? SmartTennisTV hits F1 ~97.5% largely via Kalman-smoothing noisy per-frame predictions. *(arXiv:1801.01430; MS-TCN.)* ? **reconciler/scorer.**
- **C12 ? pull ShuttleSet/ShuttleSet22 + CoachAI** as an external benchmark + weak-supervision corpus to attack the ~6-match bottleneck. *("Less is More" arXiv:1903.00859; ShuttleSet arXiv:2306.04948.)* ? **harness/eval + producers.**
- **C13 ? benchmark the rule reconciler against a learned boundary refiner** (ASRF/CBR) to *justify* staying rule-based, and ground rules in MonoTrack-style physics/cadence as **soft features, not hard rules** (hard cadence rules would re-introduce special-casing and break on doubles/lets/warm-up). *(MonoTrack arXiv:2204.01899; CBR arXiv:1705.01180.)* ? **reconciler.**

13.5 Pitfalls we are actively walking into (risk register)

- **Over-segmentation hidden by tIoU** ? the dominant error mode of cheap segment decisions; a per-window scorer can look good while fragmenting rallies. ? C4.
- **Late-fusion miscalibration / dominant-cue swamping** (over-confident shuttle CNN, low-discrimination audio). ? C2.
- **LOOCV distributional bias + tiny-N variance** ? promotion gate promotes noise without CIs. ? C1.
- **Stacking leakage** (in-sample learned-producer outputs) ? C3; and **threshold-tuning-on-test** ? keep *every* tunable knob (windowing, merge/NMS, regularization, reconciler thresholds) *inside* the LOVO loop.
- **Proposal-recall ceiling** ? the decision can't recover a rally the windowing missed; measure window recall (AR@AN) separately or you'll misattribute recall loss to the scorer.
- **Report-first decaying into silent rewrite** ? without per-rule lift gating + a confidence gate + idempotency tests, an over-eager rule systematically truncates correct long rallies.
- **Static-tripod transfer gap** ? broadcast cues (camera-switch, replays, crowd audio) that drive the tennis/soccer baselines *don't exist* on our footage; don't assume their F1s transfer.
- **Detector domain gap inherited downstream** ? WASB/TrackNet/MonoTrack are trained on other rigs; per-frame shuttle accuracy degrades off-distribution and every cue inherits that error. **Quantify detector accuracy on our footage before trusting any cue column** (currently unmeasured).

13.6 Residual gaps & honesty notes

- **Fabrications: none** across ~40 citation checks. Corrected author-lines applied above: CVIU 2009 = **Liu, Huang, Jiang, Xing, Ye, Gao** (not "Kolekar"); SmartTennisTV = **Ghosh & Jawahar**; Sensors 2024 shuttle-fusion = **Hsu, Yu, Cheng**. Metadata fixes: LOOCV-bias is **Science Advances** (10.1126/sciadv.adx6976); ShuttleSet (KDD) = arXiv:2306.04948 vs ShuttleSet22 = arXiv:2306.15664 (distinct); SF-

Net evaluates on
705 THUMOS14/GTEA/BEOID.
706 - **Read before citing** (verified to *exist*, not deep-read; both 2026, concurrent ? don't void novelty but
707 are the nearest precedents): Constraint-Aware Decoding (arXiv:2605.10149, the reconciler end-state analog)
708 and TennisExpert (arXiv:2603.13397, an audio-hits?cluster?filter rally pipeline). PCL-Former
709 (arXiv:2507.06411) ? authorship unverified; cite without asserting authors.
710 - **Weakest-evidence claims (state plainly):** warm-up-as-soft-feature (novel by *absence of counterexample*,
711 the high from-scratch-validation risk) and the measured-before-apply reconciler bundle (also absence-based).
712 - **Not yet searched:** a recent badminton full-match frame-level rally/interruption dataset (would relieve
713 scarcity); doubles-specific cue behavior; calibration for *count-valued* cues (net-crossings); whether
714 RLOOCV interacts badly with grouped folds at N=6 (the cited study is record-wise, not group-wise).
715

10. assistant (2026-06-14T06:50:45.246Z)

Now let me read the CODE_MAP.md, focusing on the parts relevant to the system/method.

11. user (2026-06-14T06:50:46.060Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-157 of 374 total (35284 tokens, cap 25000). Call Read with offset=158 limit=157 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

1 # CODE_MAP.md
2
3 **Purpose:** cold-start navigation map for the badminton-highlight-indexer ? a self-hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.
4
5 **Regenerate:** rerun the `build-code-map` workflow (parallel subsystem readers ? this doc). Re-run after any registry/contract/WSL-interface change, or after a directory restructure (this revision follows #40).
6
7 ? **2026-06-10 audit-hardening sweep changed several behaviors documented below ? trust git history over stale lines until the next full regen:** `add_video` is now an **UPSERT** (no FK-cascade wipe on re-ingest; segmentation is destroy-AFTER-confirm via `segment_watermarks`/`prune_segments`); detector artifacts are keyed by **video_id**, not basename; **`indexing.wasb.*` is now a real typed config** (was silently dropped); `timeout_sec` defaults **1800s** (was 0/hang-forever); SQLite opens with **WAL + busy_timeout**; `match_intervals` requires real overlap (iou>0); `run_regression` baseline lives in tracked `eval_baselines/` and **exits 3 on a missing baseline**; the API validates `video_path`/`output_filename`. New tests: `test_api_endpoints`, `test_compiler`, `test_wasb_runner`, `test_reingest_safety`, `test_detector_keying`, `test_config_wasb`, `test_path_safety`, `test_reliability_hardening` (+ collector `test_licensing_gate`, `test_import_guard`, `test_cvat_robustness`).
8
9 **LEGEND**
10 - `@path` = file (always repo-relative)
11 - `::sym` = symbol (class/func/const) in the nearest preceding `@path`
12 - `->` = dataflow / call / "produces"
13 - `§` = data contract (canonical shape)
14 - `?` = gotcha / footgun
15 - `?` = registry / plugin extension point
16 - `WSL` = runs inside WSL2 conda env, NOT the Windows Python process
17
18 ---

0. REPO LAYOUT ? where new files go (READ BEFORE committing a new file)

Top-level tree and the **placement rule** for each kind of file. When unsure, prefer an existing
registry / extension point (`?`) over a new top-level file.

You're adding?	Put it in?	Notes
Backend / runtime code	`backend/<subsystem>/`	`pipeline/{segmenters,detectors,validators}`, `providers/`, `sports/`, `annotations/`, `infrastructure/`, `reporting/`, `stitcher/`, `utils/`, `config/`, `features/`, `api/`. Most additions are **register a class in the relevant `?` registry**, not a new module tree.
Eval LIBRARY (importable, pure ? metrics, calibration, a scorer, a feature)	`backend/eval/`	Pure core, no side-effects at import; CLI/I-O lives in a separate driver. These are **imported widely** (e.g. `calibrate_wasb` by 8 sites) ? they are library, not scripts; do NOT move them out.
Standalone run-driver / one-off entry CLI (has `__main__`, NOT imported by app code)	**`scripts/`**	e.g. `scripts/run_phase3_eval.py`, `scripts/run_process_and_stitch.py`. Run as `python scripts/<name>.py`. If it needs `load_dotenv()`/`sys.path` before importing backend, add `"scripts/<name>.py" = ["E402"]` to `ruff.toml`.
Ops / maintenance tool (not part of the served app ? cache GC, batch admin)	`tools/`	Run as `python -m tools.<name>` (it's an importable package). e.g. `tools/cleanup_caches.py`. Keep the destructive decision path pure + unit-tested.
Training code (model fitting)	`training/`	Behind the CI-enforced import wall (ADR-008) ? `backend.main` must not import it. Own `requirements-train.txt`.
A test	`tests/test_*.py`	pytest auto-discovers; the full-suite lane has a **70% coverage floor**. Pure/offline + mocked (no GPU/WSL/network).
A doc (design, plan, living log)	`docs/`	Index it in `docs/README.md`. Only **terminal-state** (DONE/REJECTED) work moves to `docs/archives/`.
A throwaway repro / debug script	`scratch/`	**Gitignored** + excluded from ruff/pytest. Never imported by app code.
A model artifact (weights + manifest)	`artifacts/<name>/<ver>/`	ADR-008: **manifest committed** (provenance), **weights gitignored**.
Eval baseline / leaderboard JSON	`eval_baselines/`	Tracked (survives a clean checkout); the no-regression gate reads it.
Pipeline run outputs (reels, trajectories, DBs, frames)	`output/`	**Gitignored.** Per-video isolation is planned in `PER_VIDEO_OUTPUT_LAYOUT.md`.

Canonical files that stay at the repo ROOT (referenced by CI / packaging / docs ? do not move):
`setup.py` (packaging) · `run_all_tests.py` (CI: `ci.yml`) · `run_regression.py` (the no-regression gate CLI,
cited in `EXPERIMENT_HARNESS.md`) · `run_eval.py` (grandfathered ? imported by `tests/test_eval_harness.py`).
New standalone run-drivers go in **`scripts/`**, not the root.

---

1. PIPELINE

1A. Runtime: video file -> highlight reel
```
typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-defaults, never fatal)
                                          @backend/config/loader.py::load_config -> @backend/config/models.py::AppConfig
  -> video_id = MD5(whole file)           @backend/utils/hashing.py::compute_video_id (single shared helper, 64KB chunks; imported by main + all run_*.py ? no more per-entrypoint copies)
  -> validate @backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
       FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity -> SceneCut -> Blur -> Relevance   (? ACTUAL registration/exec order ? see @backend/pipeline/validators/__init__.py)
```

```
            [validation FAIL -> persist status="failed", return early; report STILL emitted in finally]
       -> db.add_video(video_id, filepath, status, [r.dict()])  @backend/infrastructure/database.py
       -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback 'motion')
       -> seg = segmenter_registry.get(seg_name)(db, global_config)   ? @backend/pipeline/segmenters/base.py::segmenter_registry  (400 + available list if missing)
       -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
            [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here without stitching]
       -> finally: emit_indexer_report(...)    @backend/reporting/__init__.py  (? ALWAYS runs ? even on validation failure / exception; NEVER raises; grafts response["reports"])
       -> select segment ids -> [(start,end)] (+ stitching padding)
       -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)  @backend/pipeline/stitcher/compiler.py
            per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy -> output/<name>.mp4
```
Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same shape):
```
P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
 -> P2 candidate "action windows" from motion/trajectory  [STOP: candidates persisted via db.add_candidate_segment]
 -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor -> provider.analyze_video(clip, prompt)   ? provider_registry
 -> P4 db.add_play_segment + db.link_candidate_to_segment
```
Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate (clamp/dedup) -> add_play_segment.
Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground truth).

1B. WASB shuttle substrate (P2 of wasb_hybrid)
```
WasbHybridSegmenter.process_video  @backend/pipeline/segmenters/wasb_hybrid.py
 -> WasbRunner.healthcheck()  (gate; not ok -> return [])  @backend/pipeline/detectors/wasb_runner.py
 -> WasbRunner.run_predict(video, out_dir)
      stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python wasb_infer.py ...'`
      -> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU detector pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
 -> TrackNetRunner.parse_trajectory_csv(csv)  @backend/pipeline/detectors/tracknet_runner.py  (shared, re-exported by WasbRunner)
 -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start, velocity_thresh, merge_gap, min_window_duration, chunk_index)  [the model-agnostic WINDOWING BRAIN]
```
(`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width` fallback (30.0, 1280.0).)

1C. Calibration / eval flow (NO pipeline run unless told)
```
GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts -> List[Interval]
DB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval]   stage ? {candidates, final}
  -> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score
  -> harness.evaluate -> EvalReport -> report_to_dict
       -> batch.aggregate (corpus micro/macro)            @backend/eval/batch.py
       -> regression.regress(_with_provider) vs baseline   @backend/eval/regression.py  [CI gate]
```

96 Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report, cross_validate} @backend/eval/calibration.py
97 Distill Gemini->local: calibrate_local (thresholds) OR distill_local (learned classifier) @backend/eval/{calibrate_local,distill_local}.py
98 Learned segmenter (offline): training.build_training_set(X,y) -> classifier.LogisticRegression.fit/save
99 Gemini refinement driver (tuning, standalone): gemini_refine.{refine_window,full_video_rallies} @backend/eval/gemini_refine.py
100 Audio probe (diagnostic only, NOT in live pipeline): audio/e1_probe.run @backend/eval/audio/e1_probe.py
101 ```
102 Entrypoints: `@run_eval.py` (single video score), `@scripts/run_phase3_eval.py` (manifest batch, can segment), `@run_regression.py` (baseline gate, exit 1=regression), `@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning drivers).
103
104 ---
105
106 ## 2. SUBSYSTEMS
107
108 ### 2.0 Orchestration & config
109 - `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS). Static UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig` (INFO, timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces stray prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI summaries may still use `print` for direct output.
110 - `::app` FastAPI; `::db_instance`,`::global_config` (`Optional[AppConfig]`) module globals set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` -> every endpoint NPEs.
111 - **ASYNC (#16, PR-1):** `::process_video` `@POST /api/process` -> fast-fail 4xx (path/exists/segmenter) -> `db.create_job` -> `_get_executor().submit(_run_job,...)` -> **202 + job_id**. `::_run_job` (worker: mark running -> `_execute_processing` -> done/failed; catches + logs traceback). `::_execute_processing` = the former sync body (hash -> validate -> add_video -> segment -> `finally:` always `emit_indexer_report`, grafts `response["reports"]`; never raises from the report); a RAISING segmenter now prune_after-restores pre-run rows before re-raising. `::get_job` `@GET /api/jobs/{job_id}` -> `{status, progress, video_id, error, result, reports, ...}` ? job `status` = EXECUTION state (`queued|running|done|failed`); the pipeline verdict lives in `result["status"]`. `::_get_executor` lazy module-global `ThreadPoolExecutor(max_workers=1)` (single box + Gemini serial-upload learning; tests monkeypatch it inline). `main()` runs `db.fail_stale_jobs()` at startup (orphaned queued/running -> failed). `::compile_highlights` `@POST /api/compile`. `::query_play_segments` `@POST /api/query`. `::get_config` `@GET /api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy `import cv2`, samples ±3s vs `indexing.motion_threshold` (dual model/dict read). ? CLI `--video-path` mode still runs the pipeline synchronously inline (separate code path, unchanged by #16).
112 - **UPLOAD/DOWNLOAD (PR-2, B2+B3):** chunked upload ? `::upload_init` `@POST /api/upload/init` (`{filename, total_size_bytes?}` -> ext allowlist + size gate -> `{upload_id}`), `::upload_part` `@PUT /api/upload/{id}/part/{n}` (raw-bytes body, STRICTLY sequential n, streams to `.partial-<id>` under `security.upload_dir`; per-part/total cap breach -> 413 + upload ABORTED), `::upload_complete` `@POST .../complete` (`{total_parts}` -> assemble -> never-overwrite rename -> `{path, video_id(MD5), size_bytes}` ? client then POSTs /api/process with that path), `::upload_abort` `@DELETE /api/upload/{id}`. ? upload state (`::_uploads` + `::_uploads_lock`) is IN-PROCESS ? restart aborts partials (client re-inits); per-user partitioning lands with PR-3. `::download_highlight` `@GET /api/highlights/{video_id}/{filename}` ? safe-name + `resolve_under_root(output_dir)` then FileResponse stream (the old /api/compile alert path was browser-unreachable). Config: `SecurityConfig.{upload_dir='output/uploads', max_upload_mb=4096, max_part_mb=95, allowed_upload_extensions=[mp4,mov]}` (95MB: free-tier edge proxies cap bodies ~100MB).
113 - `::load_config` ? ? legacy thin wrapper returning a **dict** (`load_app_config(path).model_dump(...)`); kept for transition ? new code imports `load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest` pydantic bodies.
114 - `main()` sets `global_config.output.output_dir = args.output_dir` (typed field on `OutputConfig`); `db_path = os.path.join(global_config.output.output_dir, output.db_name)`. `compile_highlights` reads the SAME `global_config.output.output_dir`, so `--output-dir` is honored end-to-end (the old write-only `_runtime_overrides` hack is gone).

115   - ? `--segmenter` argparse `choices` frozen at build time from `segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI default_segmenter fallback `'motion'`.
116   - `@scripts/run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive `input()` (blocks automation), skips validation. Config via `backend.config.load_config` (typed); segmenter, padding, and db path all read from the model (no literal fallbacks).
117   - `@run_eval.py::main` ? score one video's DB preds vs GT CSV. `--stage{candidates,final,both}`. Exit 2=arg/IO. ? pure scorer; errors if video never processed. `::_default_db_path` resolves via `backend.config.load_config` (`output.output_dir`/`output.db_name`). `get_file_md5` is an alias for `compute_video_id`.
118   - `@scripts/run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score, aggregate. `load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as label); config via `backend.config.load_config`; `segmenter_cls(db, cfg)` receives the typed `AppConfig` (same object the live pipeline feeds segmenters).
119   - `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit codes semantic: **0=ok, 1=REGRESSION (CI gate), 2=missing DB**. ? `--update-baseline` runs AFTER compare (snapshots even a regressing run); `::_default_db_path` resolves via `backend.config.load_config`.
120
121   - `@backend/config/__init__.py` ? ? canonical config import surface. Re-exports `load_config`, `save_default_config` (from `.loader`) + `AppConfig`, `Config`, `IndexingConfig`, `ValidationConfig` (from `.models`); `__all__` = exactly those 6. Use `from backend.config import load_config, AppConfig`.
122   - `@backend/config/models.py` ? Pydantic v2 models; **single source of truth for defaults**.
123   - `::AppConfig` ? root. Scalars `sport='badminton'`, `ai_provider='gemini'`, `ai_model='gemini-2.5-flash'`; sections `validation`,`indexing`,`output`,`stitching` (`default_factory`). `model_config = {extra:'allow', validate_assignment:True}`. ? `extra='allow'` -> unknown/typo'd config.json keys silently kept (legacy tolerance).
124   - ? `::AppConfig.get` ? legacy dict-compat shim: returns nested **sections as plain dicts** (`model_dump(mode='json')`, NOT the model) so chained `config.get("indexing",{}).get(...)` keeps working; falls back to searching a leaf key in any section (sections discovered dynamically from `model_fields`). ? "bit every validator on real runs" per docstring ? returning the model breaks `.get`. `::AppConfig.__getitem__` raises `KeyError` for unknown top-level keys else delegates to `.get`.
125   - `::IndexingConfig` ? segmenter/detector knobs (`motion_threshold=0.08`, `min_segment_duration=3.0`, `dead_time_merge_gap=2.0`, `chunk_duration=90.0`, `chunk_overlap=10.0`, `yolo_velocity_threshold=0.15`, `yolo_frame_skip=3`, `ai_handoff_padding=2.0`, `ai_max_workers=5`, `log/store_yolo_candidates=True`, nested `tracknet`). ? `default_segmenter='yolo_hybrid'` HERE but config.json overrides to `'gemini'`. ? `::IndexingConfig.segmenter_must_be_registered` `@field_validator` is a **no-op pass-through** ? a typo'd segmenter validates fine, only fails later at `segmenter_registry.get(...)`.
126   - `::ValidationConfig` (`min_resolution_width=1280`,`min_resolution_height=720`,`min_fps=29.9`,`min_blur_threshold=20.0`,`max_scene_cuts_per_minute=0.5`, nested `relevance`); `::ValidationRelevanceConfig` (court HSV gates, `court_pixels_min_ratio=0.05`). `::TrackNetConfig` (WSL invocation contract: `wsl_distro='Ubuntu'`, `repo_dir`, `conda_env='TrackNetV4'`, `weights_path=''`, `queue_length=5`, `velocity_threshold=0.02`). `::OutputConfig` (`default_highlights_name`, `db_name='sports_indexer.db'`, `output_dir='output'` ? CLI `--output-dir` overrides it on the typed model in `main()`). `::StitchingConfig` (`begin_padding=0.5`,`end_padding=0.5`,`use_cache=False`,`min_shot_count=1`). `::Sport` Literal of 5 sports. `::Config = AppConfig` alias.
127   - `@backend/config/loader.py` ? load/save with defaults-merge.
128   - `::DEFAULT_CONFIG_PATH = Path("config.json")` (? CWD-relative, NOT repo-root). `::_get_builtin_defaults` = `AppConfig().model_dump(...)`.
129   - `::load_config` ? precedence built-in-defaults -> file overrides. ? **shallow top-level merge** (`{**defaults, **file_data}`): a section in config.json REPLACES the whole defaults dict for that section (Pydantic then re-applies field defaults for missing leaves). Missing/unreadable/parse-error file -> fully-defaulted AppConfig + `[CONFIG WARNING]` (non-fatal).
130   - `::save_default_config` ? writes fresh `config.json`, **overwrites** if present (used by setup.py/`--setup`). `::get_default_config` ? ? transitional plain-dict alias.
131   - `@config.json` ? on-disk config; mirrors `AppConfig`. ? **`indexing.default_segmenter='gemini'`** diverges from model default `'yolo_hybrid'` ? file wins at runtime.

132 - `@setup.py` ? `::init_default_config` ? delegates to `backend.config.save_default_config` (single-source defaults; skips if config.json exists). `::run_pip_install` GPU-aware (`nvidia-smi` -> cu124 else cpu). `::run_diagnostics` (Python?3.8, ffmpeg/ffprobe on PATH, optional torch+CUDA).

133

134 ### 2.1 Segmenters (under pipeline/segmenters) ?

135 - `@backend/pipeline/segmenters/base.py`

136 - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract `name`,`description`,`process_video`. § `process_video(video_id, video_path, player_pool=None, max_frames=None) -> List[dict{id,start_time,end_time,duration,metadata}]`.

137 - `::SegmenterRegistry`, `::segmenter_registry` (singleton). ? **register a new segmenter**: subclass `BasePlaySegmenter`, call `segmenter_registry.register(MyCls)` at module bottom, AND import the module in `@backend/pipeline/segmenters/__init__.py` (registration is an IMPORT side-effect). ? `register()` builds `cls(None, {})` just to read `.name` -> name/description/`__init__` MUST NOT touch `self.db`/`self.config`. ? `list_available()` re-instantiates + swallows exceptions into "No description available." (a throwing `description` is masked).

138 - `@backend/pipeline/segmenters/__init__.py` ? imports all 6 (`motion,gemini,yolo_hybrid,tracknet_hybrid,wasb_hybrid,fusion_hybrid`) = the de-facto registration manifest; re-exports them + `segmenter_registry` via `__all__`.

139 - `@backend/pipeline/segmenters/trajectory_hybrid.py::TrajectoryHybridSegmenter` ? SHARED ENGINE for the trajectory hybrids (2026-06-11 collapse of the copy-paste twins; the "fix BOTH" footgun is retired): healthcheck gate -> P2 trajectory->windows (+persist) -> P3 provider handoff -> P4 DB/link; single `::_shot_count_from` (old wasb/tracknet shot_count drift gone). NOT registered itself. ? Subclass contract: `family` (config section `indexing.<family>.velocity_threshold(0.02)`, `output/<family>` trajectory dir, `<family>-hybrid-<model>` metadata tag), `display_label` (log lines), `name`/`description` properties, `_build_runner(idx_cfg) -> (DetectorRunner, detection_params extras)`. **Two fusion seams (IDENTITY by default ? twins byte-identical, pinned by the equivalence test): `::_enrich_candidate_stats(cw,points,fps,frame_width,video_path,idx_cfg)`** (base = the 4 motion-key stats dict; computed once into `window_stats`, used for persist + the handoff gate) **and `::_select_for_handoff(windows,window_stats,idx_cfg)`** (base = all indices). ? shared knobs still YOLO-named (`log_yolo_candidates/store_yolo_candidates`); whole-video single pass; needs `{PROVIDER}_API_KEY` (missing -> []). Equivalence pinned by `tests/test_trajectory_hybrid_equivalence.py` (detector identity, shot_count, offsets, + the base-hook-identity tests ? parametrized over both twins).

140 - `@backend/pipeline/segmenters/fusion_hybrid.py::FusionSegmenter` (name `'fusion_hybrid'`, **default-OFF** multi-signal fusion P2) ? subclasses the engine; `family='fusion'`, `display_label='Fusion'`. `::_build_runner` selects the substrate via `indexing.fusion.substrate` ('wasb'|'tracknet') reusing the existing runner+config. `::_enrich_candidate_stats` overrides to ALWAYS add `net_crossings` (Gate A signal, GPU-free via `rally_gate.gate_windows(..., estimate=self._axis_estimate(points))` ? axis/net cached per trajectory) and, only when `fusion.cue_flags.person`, the ~5 person features (`::_person_features` ? lazy `PersonDetector.detections_per_frame` over the window's ORIGINAL-video frame range ? `fusion_features.compute_person_features`). `::_select_for_handoff` overrides to apply served Gate A (`fusion.min_crossings`) ? Gate B (`fusion.min_players`, person-only) ? default 0/0 ? all windows kept. ? persisted candidates are ALWAYS the full superset (offline substrate intact); only the handoff/play_segments are gated. ? to A/B vs a tuned wasb run, set `fusion.velocity_threshold = wasb.velocity_threshold` (the engine reads `idx_cfg['fusion'].velocity_threshold`). ? person features eager even for gated windows (P3 cleanup). Tests: `tests/test_fusion_hybrid.py`.

141 - `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name `'wasb_hybrid'`, the LIVE shuttle substrate; MIT, ships pretrained badminton weights) ? thin subclass: `family='wasb'`, wires `WasbRunner`/`WasbConfig`; `detection_params` extras = `weights_path` only.

142 - `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name `'tracknet_hybrid'`) ? thin subclass: `family='tracknet'`, wires `TrackNetRunner`/`TrackNetConfig`; extras = `weights_path` + `queue_length`.

143 - `@backend/pipeline/segmenters/yolo_hybrid.py::HybridYoloSegmenter` (name `'yolo_hybrid'`, no YOLO remains; ADR-005). **Stage-1 detection EXTRACTED to `detectors/person_detector.py` (fusion P1, no behavior change)** ? the segmenter now ORCHESTRATES only (chunking, pipelined-vs-sequential P2 prefetch where ffmpeg overlaps sequential GPU, AI-provider handoff, candidate persist/link) and delegates detection to `self.person` (`PersonDetector`). Imports `_DEFAULT_DETECTOR` from person_detector for the `detection_params` snapshot. ? `yolo_tracker`/`bytetrack.yaml` is dead/back-compat; `gemini_*`

config keys are deprecated aliases still honored; still mypy-quarantined for pre-existing orchestration debt (the torchvision code that caused it left).
144 - `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `'gemini'`) ? pure-AI, no P2. Chunks are RE-ENCODED + downscaled to `indexing.ai_chunk_scale_width` (default 1280; 0 = legacy stream-copy) before upload ? stream-copied 4K chunks exceed the provider's `MAX_UPLOAD_MB` and every chunk gets refused ("0 segments / success"); re-encode also fixes keyframe drift at chunk boundaries. Oversize refusals (provider metrics `oversize_skipped`) are aggregated -> `logger.error` + `reporting.run_signals` (`oversize_clips_skipped`) for the report rule. Workers from `indexing.ai_max_workers`. ? chunks only if `duration>chunk_duration`; `max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning table).
145 - `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name `'motion'`) ? pixel absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM** (random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing server/receiver; uses `print()`; honors `max_frames`.
146
147 ### 2.2 Detectors & windowing (WSL-quarantined; ABC abstraction complete)
148 - `@backend/pipeline/detectors/base.py` ? DetectorRunner ABC (healthcheck() -> Tuple[bool,str] never-raises; run_predict(...) -> Optional[str] CSV path). Contract deliberately matches the pre-existing tuple returns and call sites in the hybrids. Rich module docstring includes "how to add a new runner", Linux-native path notes (the de-risk goal), and guidance on threading health into reports. Re-exported from `detectors/__init__.py`. This finishes the WSL de-risk seam (low-regret 6 / review item 1).
149 - `@backend/pipeline/detectors/tracknet_runner.py`
150 - `::TrackNetConfig`(+`.from_indexing_cfg`; ? key drift `wsl_distro` JSON -> `distro` field), `::TrackNetRunner` ? runs TrackNet `predict.py` in WSL.
151 - `::to_wsl_mnt_path` (`C:\x`->`/mnt/c/x`; ? POSIX/`~` passthrough, UNC unhandled), `::TrajectoryPoint`(frame,visible,x,y).
152 - **`::parse_trajectory_csv` @staticmethod** + **`::trajectory_to_action_windows` @staticmethod** = MODEL-AGNOSTIC brain, re-exported verbatim by WasbRunner. ? `weights_path` defaults `''` -> `run_predict` returns None; predict.py only `.mp4/.avi` + hard-names `<stem>_predict.csv`; `visible==False` resets prev (occlusion gap breaks track); `fps<=0->30`, `frame_width<=0->1280` silent; `track_id_count` hardcoded 1; `timeout_sec=0` -> NO timeout (hangs forever). `::_stage_video` "OK" sentinel.
153 - `@backend/pipeline/detectors/wasb_runner.py::WasbRunner` (inherits DetectorRunner) (+`::WasbConfig`, from `indexing.wasb`; ? `weights_path` HAS a real default `wasb_badminton_best.pth.tar` ? no required guard) ? stages `wasb_infer.py` (`::_INFER_WIN`) + video into WSL, runs in `wasb` env. `::parse_trajectory_csv`/`::trajectory_to_action_windows` = `staticmethod(TrackNetRunner.*)` (explicit rebind = the plugin-equivalence point). Output hard-named `<stem>_wasb.csv`. ? `wasb_infer.py` re-staged each run (editing Windows copy inert until staged); `_wsl_bash` now shared via `detectors/base.py::WslCommandMixin` (2026-06-11 dedupe). Both runners now satisfy the DetectorRunner ABC for future swappability. **Cache hygiene (#109):** on a SUCCESSFUL run both runners delete their WSL intermediates ? WASB the frame cache + staged video, TrackNet the staged video ? gated by `indexing.{wasb,tracknet}.keep_frames` (default false; on FAILURE kept so a re-run resumes). Pre-run `min_free_gb` disk-guard warning. Shared helpers `WslCommandMixin::_wsl_rm_rf`/`_wsl_free_gb`.
154 - `@backend/pipeline/detectors/wasb_infer.py` (WSL) ? `::run` core. ? imports WASB-SBDT repo modules + torch ? NOT importable on Windows. Reboot-resumable detector cache: `::_DET_FILE='det_raw.jsonl'`,`::_MANIFEST_FILE='manifest.json'`, `::CACHE_VERSION=1`, `::load_and_clean_cache` (reads longest CONTIGUOUS prefix where `w==line index`, rewrites to heal crash-truncated tail), `::manifest_compatible` (? `frames_dir` abspath'd but `weights` compared raw), `::_atomic_write_text/_atomic_write_trajectory`, `::_dets_to_tracker` (xy MUST be np.array), `::_build_cfg` (`runner.gpus=[0]` hardcoded), `::extract_frames` (cv2-PNG SLOW), `::default_cache_dir` (`<stem>_wasbcache` next to out). ? GPU pass resumable; CPU tracker STATEFUL -> always fully re-run; `--fresh` ignores cache.
155 - `@backend/pipeline/detectors/person_detector.py::PersonDetector` ? the **PERSON rally cue** (torchvision detector + supervision ByteTrack + velocity windowing), extracted from yolo_hybrid (fusion P1; type-clean, NOT quarantined). `::_PERSON_CLASS_ID=1` ? (ultralytics used 0; wrong value -> zero detections), `::_DETECTOR_FACTORIES`/`::_DEFAULT_DETECTOR='fasterrcnn_resnet50_fpn'`. `::lazy_load_model` (lazy `torchvision` import so module/tests load without it; `self.model` typed `Any`), `::make_tracker` (per-chunk ByteTrack ? IDs chunk-local), `::detect_and_track` (one frame -> confident persons w/ track IDs; BGR->RGB, COCO person filter, ByteTrack association), `::extract_action_windows_from_chunk` (chunk -> high-motion windows w/ `peak/mean_velocity`,

`active_frame_count`, `track_id_count` ? the stats that feed candidate `stats` AND the future learned-fusion person features). ? velocity at `effective_fps = fps/frame_skip` -> `yolo_velocity_threshold` coupled to `frame_skip`. Consumed by yolo_hybrid AND the fusion segmenter (P2 reuses the SAME producer over shuttle-seeded windows via `::detections_per_frame` ? add-a-producer, not a refactor). BSD torchvision + MIT ByteTrack.
`::detections_per_frame(video_path,start_frame,end_frame,frame_skip)` (fusion P2) reads the ORIGINAL video by ABSOLUTE frame range (seek+sequential) ? `{frame_idx: [(track_id,bbox)]}` + `fps/frame_width/frame_height`, so indices align 1:1 with the WASB trajectory `.frame` (no clip re-timing). ? cv2 frame-seek accuracy varies by codec (±a few frames on real footage).
156   - `@backend/pipeline/detectors/fusion_features.py` ? **pure** per-window person-cue feature math (no I/O/torch/cv2; mirrors rally_gate discipline). Inputs: per-frame `{frame_idx:[(tid,bbox)]}` + the in-window shuttle points + optional normalized `court_polygon` + `net=(axis,pos)`. `::point_in_polygon` (ray-cast), `::person_in_court` (foot-point; None polygon=count-all; bad dims=False), `::in_court_counts`, `::players_in_court` (median), `::in_court_ratio`, `::mean_player_motion` (per-track centre displacement; x by width, y by height), `::two_sided_motion` (both net-halves occupied), `::shuttle_player_colocation` (shuttle inside a person box ±`pad_frac` halo over aligned visible frames ? **the held-shuttle discriminator**), `::compute_person_features` (aggregator ? the 5 `::PERSON_FEATURE_NAMES`). All scale/translation-invariant, graceful to None/empty. Tests: `tests/test_fusion_features.py`.
157   - `@backend/pipeline/detectors/rally_gate.py` ? Gate A net-crossing post-filter (pure, no I/O/model). `::estimate_play_axis` (larger-spread axis; ? ties -> 'x'), `::count_net_crossings` (sign changes vs median net; deadband jitter zone), `::gate_windows`/`::apply_gate` (`min_crossings=2` default; kills single-toss service feeds; **`estimate=` param** lets a caller scoring many windows pass a precomputed `(axis,net,span)` to skip per-window re-estimation ? used by FusionSegmenter). `::GatedWindow`(start,end,crossings,visible_points,kept). ? window contract here is `(start,end)` tuples NOT the rich window dict ? caller must adapt. Consumed by calibrate_local / distill_local / export_wasb_segments / fusion_hybrid.

12. assistant (2026-06-14T06:50:54.308Z)

I have enough from CODE_MAP §1-2 plus the four design docs to synthesize the answer. The eval-stack details (LOVO, regression, classifier) are confirmed in the harness doc's audit table. Let me confirm the eval-stats helpers exist (C1-C4 wiring) since the harness doc references `backend/eval/eval_stats.py` and `segmentation_metrics.py`.

13. user (2026-06-14T06:50:54.589Z)

backend\eval\eval_stats.py
backend\eval\segmentation_metrics.py

14. assistant (2026-06-14T06:50:58.784Z)

Both helper modules exist, confirming the C1/C4 measurement-honesty wiring is real and shipped (consistent with the recent commits #124-#127). I have everything needed. Let me return the structured output.

15. user (2026-06-14T06:52:21.037Z)

Structured output provided successfully

16. assistant (2026-06-14T06:52:31.961Z)

I have extracted the SYSTEM + METHOD + measurement-honesty framework from all five docs and returned it via StructuredOutput.

Summary of what I captured:

- **2-layer mental model**: frozen WASB shuttle-trajectory substrate (DETECTION) ? velocity ? shuttle-seeded candidate windows (the model-agnostic `trajectory_to_action_windows` "windowing brain") ? a learned numpy LR scorer (SEGMENTATION/DECISION) ? `[start,end]` rallies graded by temporal IoU. Golden labels grade Layer 2 only (times, not coordinates).

- **Signals as confidence-weighted feature columns**: every producer emits `(value, confidence)` persisted to `candidate_segments.stats` as `<producer>__<key>`; the LR scorer *learns to weight* them ? no `if/else` branches, default-OFF cue flags, promote-only-on-measured-LOVO-lift, rollback = config flip. The detection?decision persist-once/re-score-forever substrate makes experiments zero-risk offline re-scores.

- **Measurement-honesty C1?C4**: C1 bias-aware LOVO (group-by-match) + per-fold CIs + bootstrap 95% CI + paired sign test (shipped in `eval_stats.py`, commits #124/#127); C2 per-producer calibration; C3 out-of-fold stacking; C4 segmental F1@k + edit + segment-count ratio because tIoU is blind to over-segmentation (`segmentation_metrics.py`). Promotion rule: lift is real only when ?F1 CI clears 0 AND the sign test agrees.

- **Honest related-work framing (§13)**: skeleton = Temporal Action Localization/Segmentation + late/score-level (stacked) fusion = standard prior art ? "cite, don't claim"; only the report-first reconciler-as-linter bundle rated `novel-combination`; warm-up-as-soft-feature is the one thin genuine gap; zero fabricated citations across ~40 checks.

All file paths are absolute. No files were written.